

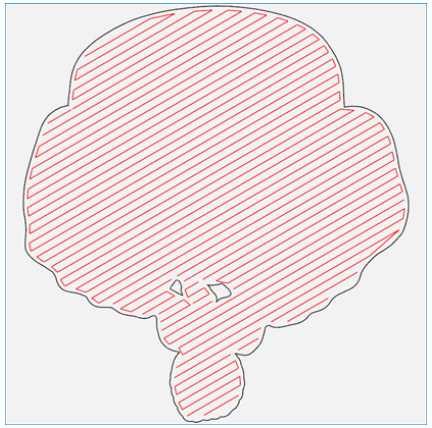
计算机辅助制造实践

——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟
玉泉教1-215　zjtzhylin1986@163.com

第9章 方向平行填充路径生成

- 扫描线法填充线生成
- 基于裁剪的填充线生成
- 填充区域分区
- 平行路径生成与连接
- 最少跳刀填充方向优化
- 平行路径生成测试





1. 扫描线法填充线生成

平行填充的核心是平面区域的平行线填充算法

即用一组具有一定倾斜角度的**等距**平行线去截交区域边界，计算并统计落在区域内部的有效线段。

- **算法输入**：**平面多边形区域**和**填充角度**，区域包含外边界（逆时针）和内边界（顺时针）。
- **基本原理和步骤**：

①生成一组指定倾角且能够覆盖整个区域的平行直线序列；

②遍历其中每条直线，计算当前直线和区域的交点；

③根据交点的在直线上的奇偶顺序提取填充线段并输出。

注：

步骤3中，在每条直线上，如果交点序号从0开始计数，那么从**偶数到奇数**序号交点连成的线段将落在区域内部，即这些线段是**有效**的填充线段。



平面区域内平行填充线生成原理

1. 扫描线法填充线生成

①生成一组指定倾角且能够覆盖整个区域的平行直线序列；

②遍历其中每条直线，计算当前直线和区域的交点；

③根据交点的在直线上的奇偶顺序提取填充线段并输出。

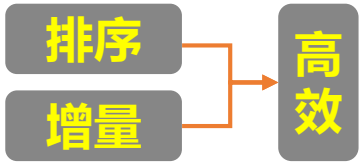
- 以上步骤的核心是**平行线**和**多边形**的**求交计算**。
  - 如果不计时间成本，直接暴力求交，则直线和线段的总求交次数将达m　n，其中m为平行线数量，n为多边形总顶点数。
- **任务**：**平行线和多边形求交，尽量降低求交计算量！**
- **方法**：**扫描线法。**
  - 平行线和多边形的求交问题和STL模型的截交问题十分相似，STL截交使用了扫描平面（sweep plane）法来加快截交计算的速度。仿照扫描平面法的框架，实现一种用于平面平行线和多边形间的扫描线求交算法。



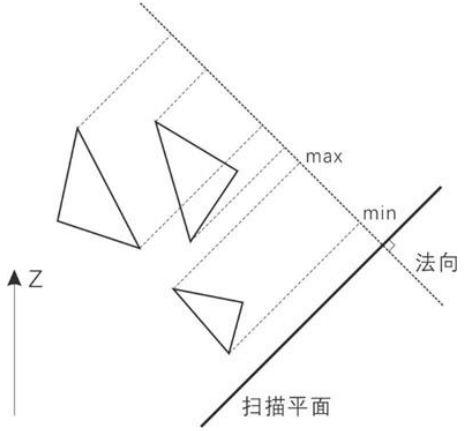
平面区域内平行填充线生成原理

1. 扫描线法填充线生成

- **扫描线数据结构：扫描平面法回顾**
- 扫描平面法对STL模型进行快速截交包含**3个要点**：
  1. 扫描平面**数据结构**建立；
  2. 对STL模型中的三角面片按照面片最低点的z坐标从小到大进行**排序**；
  3. 当扫描平面移动时**剔除**不再与之相交的面片，同时从STL模型中**添加**新的与之相交的面片。



- 在切片应用中，**扫描平面始终和坐标系Z轴垂直**。
  - 在这种情况下，STL模型中的各个三角面片相对于扫描平面法向的**最低点最容易求取**，只需直接比较面片顶点的z坐标即可。
  - 如果扫描平面和Z轴不垂直，如图，则需要把三角面片各顶点投影到扫描平面的法向上，并在法向上找出顶点投影的最低点。这无疑会增加扫描平面法的实施难度，而且扫描平面法将变得更加难以理解。



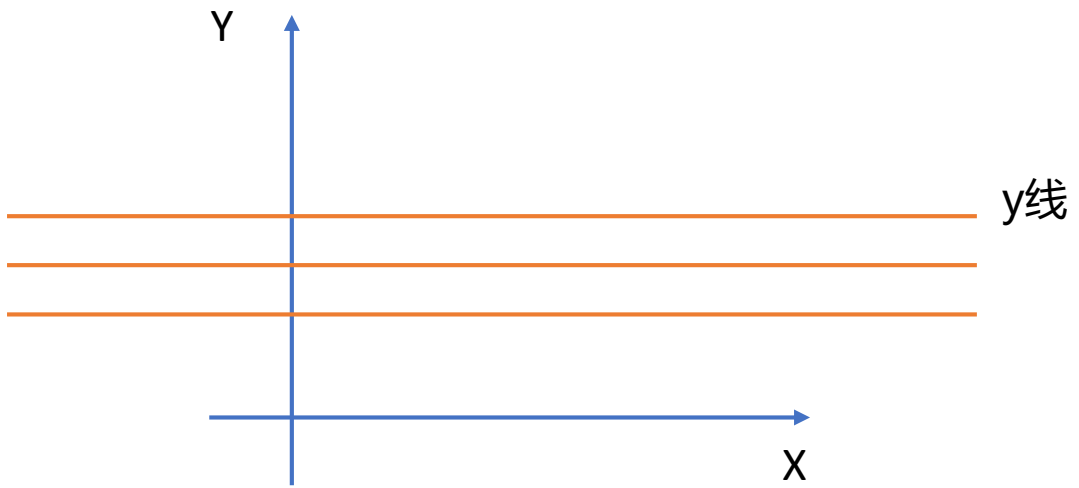
扫描平面不和Z轴垂直的情况

问题？

在平面区域的填充应用中，平行线（扫描线）方向是不确定的！

1. 扫描线法填充线生成

- **扫描线数据结构**
- 针对平面（指XY平面）平行线和多边形的交点计算，为了降低难度，**假设平行线相对于X轴的角度为0**。在这种情况下，一条直线只需用它在Y轴上的**截距（y值）**就可以表示，称截距为该直线的**高度**，称直线本身为**y线**。
  - 假设每条平行线的高度已知，被保存到一个实数序列 ys 中。ys 可根据指定的平行线间距生成，也可由外部输入。
- 仿照扫描平面法，建立扫描线数据结构，类名为**SweepLine**，写在**GenHatch**模块中。
  - 扫描平面的作用是维护和当前高度扫描平面相交的三角面片序列；而在平面上，扫描线的作用是维护和当前高度扫描线相交的线段序列。



1. 扫描线法填充线生成

- **扫描线数据结构**

GenHatch.py	GenHatch模块，SweepLine（扫描线）类实现	9-1

```
1   class SweepLine:                                    # 定义SweepLine类，扫描线
2       def __init__(self):
3           self.segs = []                              # 属性segs，Segment类型列表
4       def intersect(self, y):                         # 定义segs和y线的求交函数
5           ips = []                                    # 交点列表
6           yLine = Line(Point3D(0, y, self.segs[0].A.z), Vector3D(1, 0, 0)) # 创建和X轴平行的y线
7           for seg in self.segs:                       # 遍历segs，和y线求交
8               if seg.A.y == y:                        # 重要：如果y线经过线段A端点
9                   ips.append(seg.A.clone())           # 则将A端点添加至交点列表
10              elif seg.B.y == y:                      # 同理，如果y线经过线段B端点
11                  ips.append(seg.B.clone())           # 则将B端点添加至交点列表
12              else:                                   # 否则，使用intersect函数计算y
13                  ip = intersect(yLine, seg)          # 线和线段交点，并将交点添加至
14                  if ip is not None:                  # 交点列表，ip：intersection point
15                      ips.append(ip)
16          ips.sort(key=lambda p: p.x)                 # 对交点按x值从小到大排序
17          i = len(ips) - 1
18          while i > 0:                                # 移除交点列表中的重合点对
19              if ips[i].distanceSquare(ips[i-1]) == 0:    # 逐个比较相邻点是否重合
20                  del ips[i]
21                  del ips[i-1]
22                  i = i – 2                           # 如果点对被移除，i向前走2步
23              else:                                   # 否则i向前走1步
24                  i = i - 1
25          return ips                                  # 返回处理后的交点列表
```

重要：移除重合交点对

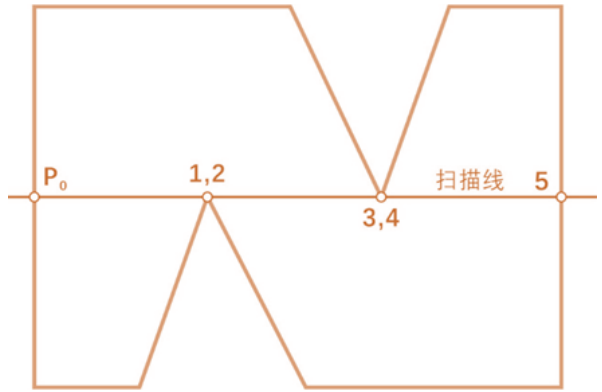
1. 扫描线法填充线生成

- **扫描线数据结构**
- **segs** 属性，用于存储和当前扫描线相交的线段。
- **intersect** 函数作用是计算扫描线维护的线段和指定y直线的所有交点。在这个函数中，首先根据输入的y值构造了一条y线，然后遍历segs中的每条线段和y线求交。

  - 理论上，只要调用GeomAlgo模块中的intersect函数去计算直线（Line）和线段（Segment）的交点即可。但当线段端点恰好落在直线上时，由于浮点运算中截断误差的存在，intersect函数可能无法返回正确的结果，在最差的情况下，intersect函数可能返回None，即直线和线段在本应相交的情况下却返回不相交的错误结果。在求平面区域填充线的应用中，某处交点的有无将直接改变当前y线上所有交点的奇偶属性，影响填充结果。
  - 为了增强算法的稳定性，首先判断线段两端点是否落在直线上，当直线是一条和X轴平行的y线时，这样的判断变得非常简单，只需判断线段端点的y值是否和y线的y值相等即可。如果线段端点没在直线上，再调用GeomAlgo模块的intersect函数求取直线和线段交点。
  - 在获取所有交点之后，对交点列表按x坐标进行了升序排序，目的是方便后续直接从交点列表中提取填充线。此外，还执行了一步看似奇怪的操作：通过检查相邻交点间的距离平方是否为0，删除交点列表中所有重合的交点对（**为什么？**）。

注意：在intersect函数中删除的重合交点对不会对填充结果的正确性造成影响。

- 如图，重合交点对只可能存在以下两种情形：一是交点两边的线段都在y线下方（图中P_1、P_2），二是交点两边的线段都在y线上方（图中P_3、P_4），无论哪种情况，因为两边线段都和y线相交，因此最终会计算得到两个重合的交点。**为什么只有这两种情形?**
- 在图中，按照填充线的提取规则，最终将得到 P_0P_1、P_2P_3、P_4P_5 三条有效填充线段。考虑到 P_1、P_2 重合，P_3、P_4 也重合，而且所有交点都在同一条直线上，因此可移除 P_1、P_2 和 P_3、P_4 两组点对，得到和原来3条填充线等价的 P_0P_5 填充线。
- 根据以上分析，删除重合的交点对对最终填充结果的正确性不产生影响，而且可以减少输出的填充线数量。



1. 扫描线法填充线生成

- **填充线交点计算**
- **扫描线法关键步骤：**一是根据线段最低点的y坐标对线段序列进行排序；二是当扫描线平移时对其中维护的线段进行增删。参考扫描平面法框架，在GenHatch模块中实现**calcHatchPoints**函数。

GenHatch.py | **GenHatch模块，calcHatchPoints函数实现，用于计算平行线和多边形交点** | **9-2**

```
1   ……                                          # 导入相关库，省略其他代码
2   def calcHatchPoints(polygons, ys):          # 输入多边形和扫描高度ys
3       segs = []                               # 存放多边形边的线段列表
4       for poly in polygons:                   # 遍历多边形，收集所有线段
5           for i in range(poly.count() - 1):
6               seg = Segment(poly.point(i), poly.point(i + 1))   # 根据多边形相邻顶点创建线段
7               seg.yMin = min(seg.A.y, seg.B.y)                  # 计算并保存线段最低点y值yMin
8               seg.yMax = max(seg.A.y, seg.B.y)                  # 计算并保存线段最高点y值yMax
9               segs.append(seg)
10      segs.sort(key=lambda s: s.yMin)         # 根据最低点y值对线段升序排序
11      k, sweep = 0, SweepLine()               # 创建扫描线对象
12      ipses = []                              # 交点列表
13      for y in ys:                            # 遍历扫描线高度
14          for i in range(len(sweep.segs) - 1, -1, -1):   # 移除不再和扫描线相交的线段
15              if sweep.segs[i].yMax < y:
16                  del sweep.segs[i]
17          for i in range(k, len(segs)):       # 从segs添加和扫描线相交的线段
18              if segs[i].yMin < y and segs[i].yMax >= y:   # 将满足最低点小于y，最高点大于
19                  sweep.segs.append(segs[i])  # 等于y的线段添加至扫描线
20              elif segs[i].yMin >= y:         # 知道线段最低点大于等于y值，
21                  k = i                       # 停止添加线段，并记录k值
22                  break
23          if len(sweep.segs) > 0:             # 如果扫描线中线段数量不为0，
24              ips = sweep.intersect(y)        # 则调用SweepLine的intersect
25              ipses.append(ips)               # 函数计算扫描线和线段交点
26      return ipses                            # 返回得到的二维交点列表
```

k是扫描线法高效的关键所在，如果没有这个变量，对每个扫描高度，不得不从线段列表segs的首个元素开始遍历，这样对segs的排序就显得毫无意义。

1. 扫描线法填充线生成

- **填充线交点计算**

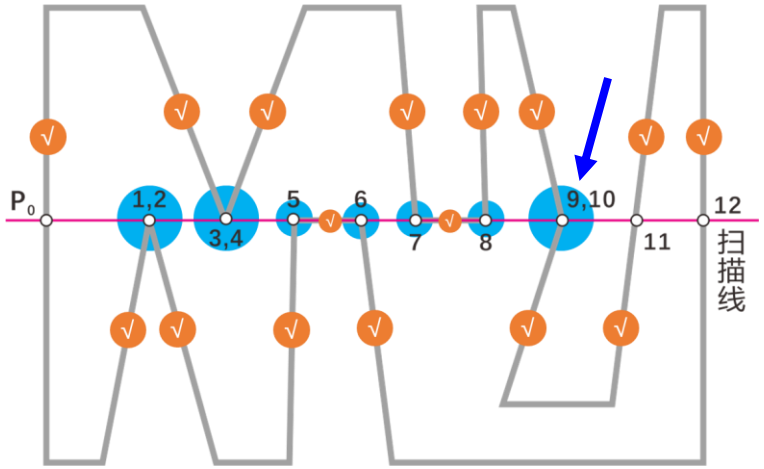
- **calcHatchPoints**函数的输入包含两个参数，第一个参数是多边形列表polygons，第二个参数是扫描高度列表ys。根据填充线的提取原理可知，对这个函数而言，多边形的方向是不重要的，并不一定要求满足“外逆内顺”的条件。

- **calcHatchPoints**函数实现流程：
  - 首先，收集所有多边形的边线段，然后根据每条线段yMin值对线段从低到高排序。
  - 对当前扫描高度y，首先从扫描线中移除已经不再和扫描线相交的线段，即那些yMax值小于y的线段；然后从segs中向扫描线添加和扫描线相交的线段，即那些yMin值小于y且yMax大于等于y的线段；在这个过程中，如果当前线段的yMin大于等于y值，则停止向扫描线添加线段，同时记录当前线段在segs列表中的**序号k**。
  - 最后，使用SweepLine类的intersect函数计算扫描线中线段和扫描线的所有交点，并将这些交点保存到ipses中，这是一个二维列表，其中第一个维度表示扫描高度序号，第二个维度表示某一扫描高度上的所有交点。

- **calcHatchPoints**函数中的**向扫描线添加新线段的条件：**
  - **条件1：yMin < y且yMax ≥ y**。在代码中，对某一扫描高度y，仅将满足条件1的线段添加至扫描线，即小于时等号取不到，大于时等号能取到。
  - **条件2：yMin ≤ y且yMax ≥ y**。为什么条件不是条件2，小于大于同时取到等号呢？这样看起来更加合理。

1. 扫描线法填充线生成

- **填充线交点计算：添加新线段的条件**
- 如果使用**条件2**，只要某线段落在 y 线上，就将该线段添加至扫描线，包括：**（1）线段上端点落在 y 线上、（2）下端点落在 y 线上、（3）中间被 y 线穿过，**以及**（4）上下两端点都落在y线上**（即线段和y线重合）4种情况。
  - 从某种意义上来说，应用条件2是合情合理的，它会收集所有和当前y线相交的线段。但是，当y线经过线段端点时，由于线段的连续性，重合的端点将被收集两次，这无疑会增加后续根据交点奇偶属性来提出填充线的难度。
  - 条件2并不会影响图中P_1、P_2，P_3、P_4，P_5、P_6以及P_7、P_8这4组点对在提取填充线时的结果，即便保留这些点对，最终也能提取到几何上正确的填充线，因为在这些点上，y线并未真正“穿过”轮廓。
  - 但是，条件2会影响图中P_9、P_{10}在提取填充线时的结果，如果不加处理，将提取到位于多边形区域外的填充线（即线段$P_{10}P_{11}$），这显然是错误的。
  - 需要说明的是，如果线段和直线重合，认为它们不相交，这也是为什么图中在P_5、P_6、P_7、P_8处只有一个点的原因。



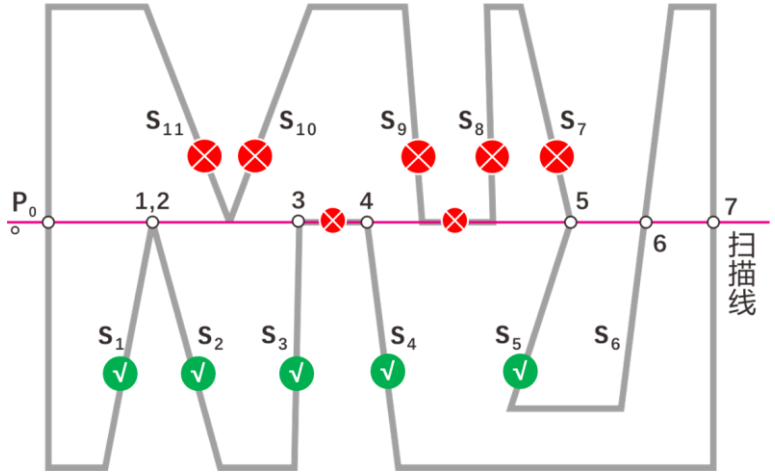
根据条件2获得的扫描线上的交点

？

如何修补条件2

1. 扫描线法填充线生成

- **填充线交点计算：添加新线段的条件**
- 使用条件1，即当线段满足**yMin < y且yMax ≥ y**时，才将其添加至扫描线。
  - 首先，使用这个条件会自然排除那些和y线重合的线段。
  - 其次，如图，同样对那些端点落在y线上的线段，使用条件1会为扫描线筛出那些位于y线下方的线段，即图中线段 s_1~s_5；排除那些位于y线上方的线段，即图中线段 s_7~s_{11}。在这种情况下，可以得到初始交点序列 P_0~P_7。
  - 在P_5处，自然而然地仅生成了一个符合正确填充逻辑的交点，这在使用条件2时是难以想象的。
  - 使用条件1，生成了一些重合的交点对，如的 P_1、P_2，但是这并不会对填充结果的正确性造成影响。而且，考虑到使用了SweepLine的intersect函数计算y线和线段的交点，会从初始得到的交点序列中删除重合的交点对，因此最终将得到P_0、P_3、P_4、P_5、P_6、P_7这6个交点，生成P_0P_3、P_4P_5、P_6P_7这3条填充线段。



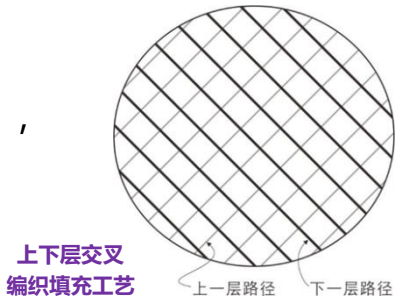
根据条件1获得的扫描线上的交点

一个等号对算法结果产生了巨大影响

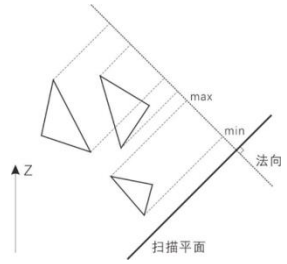
在设计判断条件时需要深思熟虑
哪怕这个条件看起来非常简单!

1. 扫描线法填充线生成

- **填充线方向调整**
- **三维打印工艺特点**：打印过程中要求上下层切片间的平行填充路径能够呈一定的角度（如90度、120度等），在上下层材料之间形成相互交错的**编织**效果，以增加层间材料连接强度。
  - 但目前只能为多边形生成和 **X 轴平行**的填充线，即填充线的倾角 **θ 只能为 0 度**。



思路一、改变扫描线的角度



改变扫描线方向，并将所有线段投影到扫描线的法线方向进行排序。

- 这种思路虽然直观，但会增加算法的实现难度；
- 且当扫描线不和X轴平行时，对线段端点是否落在扫描线上这一临界情形进行判断的数值稳定性将大大下降，这对后续填充线的提取是致命的。

——不可行思路

思路二、对输入多边形进行旋转



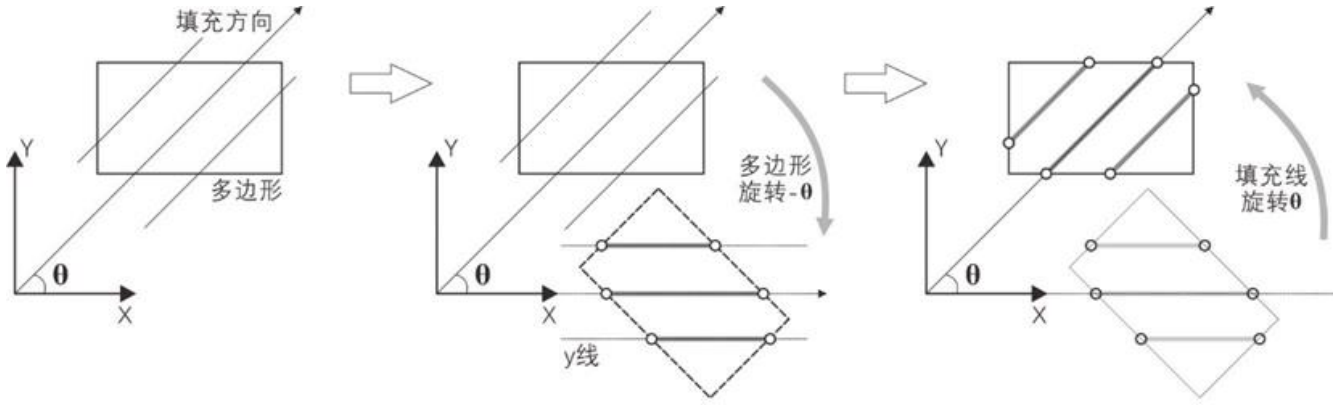
相对运动：**要么旋转扫描线，要么反向旋转多边形，它们的相对位置关系不变！**

- 对旋转后的多边形，可以使用已实现的填充线生成算法，无需设计新的算法。
- **反向**旋转多边形，生成填充线，**正向**旋转填充线。

——可行思路

1. 扫描线法填充线生成

- **填充线方向调整：原理和步骤** ——旋转多边形，计算填充线交点，旋转填充线段
- 对输入的多边形绕平面坐标系原点旋转 –θ 角度，其中负号表示旋转方向为顺时针。
- 对得到的多边形，根据填充线的间距生成一组高度序列 ys。
- 将**旋转多边形**和 ys 输入已实现的 **calcHatchPoints** 函数，生成填充线交点。
- 对得到的交点，按照填充线提取原则将其转化为填充线段。此时所有的填充线段都和X轴平行，而且它们没有落在正确的位置上。
- 将得到的填充线段绕坐标系原点**旋转θ角度**，让其落在原多边形内部。旋转后的填充线段就是要求的填充线段，它们相对于X轴正向的倾角恰好为θ。



(a) 多边形和填充角度　(b) 对边形旋转-θ 角度　(c) 对填充线旋转 θ 角度

根据条件1获得的扫描线上的交点

？

如何旋转
多边形和线段

1. 扫描线法填充线生成

- **填充线方向调整：多边形和线段旋转**
- **Polyline类扩展：**增加旋转函数，输入旋转矩阵m

Polyline.py | **多段线类，增加multiply和mulitipled两个函数，以变换多边形** | **9-3**

```
1   class Polyline:
2       ……                                  # 省略Polyline中其他函数
3       def multiply(self, m):              # 对对象本身变换，作用一个矩阵m
4           for pt in self.points:
5               pt.multiply(m)
6       def multiplied(self, m):            # 作用矩阵m，产生一个新的多段线对象
7           poly = Polyline()
8           for pt in self.points:
9               poly.addPoint(pt*m)
10          return poly
```

- **Segment类扩展：**增加旋转函数，输入旋转矩阵m

Segment.py | **线段类，增加multiply和mulitipled两个函数，以变换线段** | **9-4**

```
1   class Segment:
2       ……                                  # 省略Polyline中其他函数
3       def multiply(self, m):              # 对对象本身变换，作用一个矩阵m
4           self.A = self.A.multiplied(m)
5           self.B = self.B.multiplied(m)
6       def multiplied(self, m):            # 作用矩阵m，产生一个新的线段对象
7           seg = Segment(self.A, self.B)
8           seg.multiply(m)
9           return seg
```

1. 扫描线法填充线生成

- **填充线方向调整：**在GenHatch模块中实现**genSweepHatches**函数，输入多边形，输出任意指定倾角填充线。

GenHatch.py | **GenHatch模块，genSweepHatches函数实现，生成指定倾角的均匀填充线** | **9-5**

```
1   def genSweepHatches(polygons, interval, angle):      # 定义均匀填充线生成函数
2       mt = Matrix3D.createRotateMatrix('Z', -angle)    # 旋转矩阵，matrix to，去
3       mb = Matrix3D.createRotateMatrix('Z', angle)     # 旋转矩阵，matrix back，回
4       rotPolys = []                                    # 存储旋转后多边形
5       for poly in polygons:                            # 遍历，旋转多边形，作用mt
6           rotPolys.append(poly.multiplied(mt))
7       yMin, yMax = float('inf'), float('-inf')         # 获取旋转后多边形最低和最高值
8       for poly in rotPolys:
9           for pt in poly.points:
10              yMin, yMax = min(yMin, pt.y), max(yMax, pt.y)
11      ys = []                                          # 存储扫描线高度
12      y = yMin + interval                              # 其实高度
13      while y < yMax:                                  # 根据输入间距生成高度列表ys
14          ys.append(y)
15          y += interval
16      segs = genHatches(rotPolys, ys)                  # 对旋转多边形生成填充线段
17      for seg in segs:                                 # 遍历，旋转填充线段，作用mb
18          seg.multiply(mb)
19      return segs                                      # 最后返回旋转后的填充线列表
20  def genHatches(polygons, ys):                        # 填充线生成函数，不涉及旋转
21      segs = []
22      ipses = calcHatchPoints(polygons, ys)            # 生成填充线交点，二维列表
23      for ips in ipses:                                # 根据填充线奇偶提取原则从交点
24          for i in range(0, len(ips), 2):              # 列表中提取填充线
25              seg = Segment(ips[i], ips[i + 1])
26              segs.append(seg)
27      return segs                                      # 返回填充线段，一维列表
```

旋转多边形

生成 ys

1. 扫描线法填充线生成

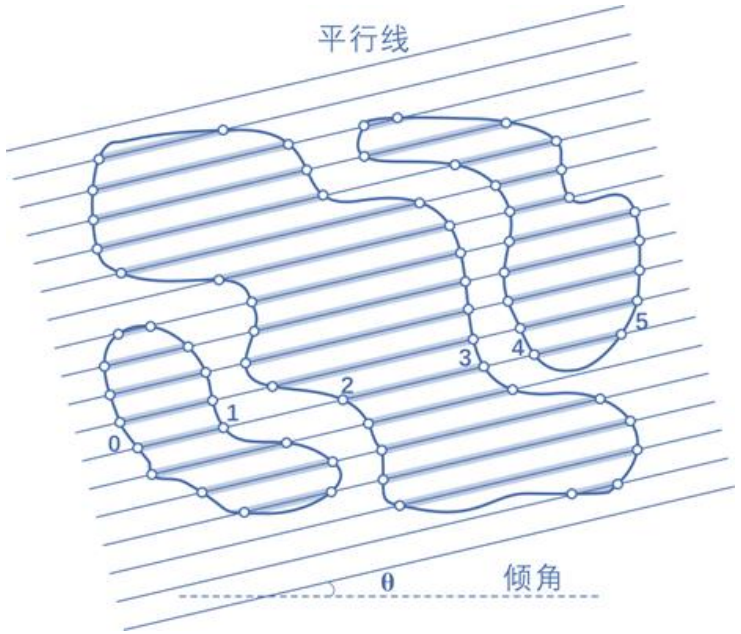
- **填充线方向调整：实现**
- **genSweepHatches**函数输入包含3个输入参数，其中
  - polygons　　需要被填充的多边形
  - interval　　填充间距
  - angle　　填充线相对于平面坐标系X轴的倾角
- 在genSweepHatches函数中：
  - 首先创建了两个旋转矩阵变量，mt和mb，其中mt用于将几何实体顺时针旋转指定角度，而mb用于将几何实体逆时针旋转指定角度。
  - 然后对输入多边形列表中的每个多边形作用矩阵mt，得到旋转多边形列表。
  - 在生成均匀的扫描线高度列表ys后，通过调用**genHatches**子函数生成了相对于旋转后多边形的填充线。
  - 最后，对这些填充线作用矩阵mb，得到目标多边形的填充线。
- **genHatches**函数的作用是根据输入多边形和扫描高度列表生成填充线段列表。其核心是调用calcHatchPoints生成多边形和平行线的交点列表，然后根据填充线奇偶提取原则将交点列表转化为填充线列表。
  - genHatches函数和calHatchPoints函数的输入一样，不包含填充线的倾角，因此它也只能返回和X轴平行的填充线。
  - genHatches函数输出的填充线列表是一个**一维列表**，它失去了扫描线的层次信息，无法从这个返回列表中直接判断两条填充线段是否在同一条扫描线上（当然通过比较线段坐标信息还是可以间接判断的）。

2. 基于裁剪的填充线生成

- **平行线段生成原理**
- 换个角度观察填充线和平行线以及多边形间关系，求取填充线的过程其实就是以多边形为窗口对平行线进行裁剪的过程。——裁剪正是**Clipper库**最擅长做的事！
  - Clipper支持四种裁剪操作：**交（intersection）**、**并（union）**、**差（difference）**、**异或（xor）**。
  - Clipper除了能够处理封闭轮廓间的裁剪问题外，还**支持封闭轮廓和开放轮廓的求交裁剪操作**，这一功能正好满足生成填充线的需求。
- 对输入的多边形区域，使用Clipper库在其内部生成平行填充路径的步骤如下：
  - 步骤1. **生成一组指定倾角且能覆盖整个区域的平行线段序列**；
  - 步骤2. 以多边形区域为窗口，对平行线进行裁剪，输出填充线。

✓ 使用Clipper库，虽然不用再去旋转多边形，但需要输给它实实在在的**带倾角的线段对象**。因为Clipper库只能处理多段线对象（线段也是一种多段线），而无法处理直线对象。

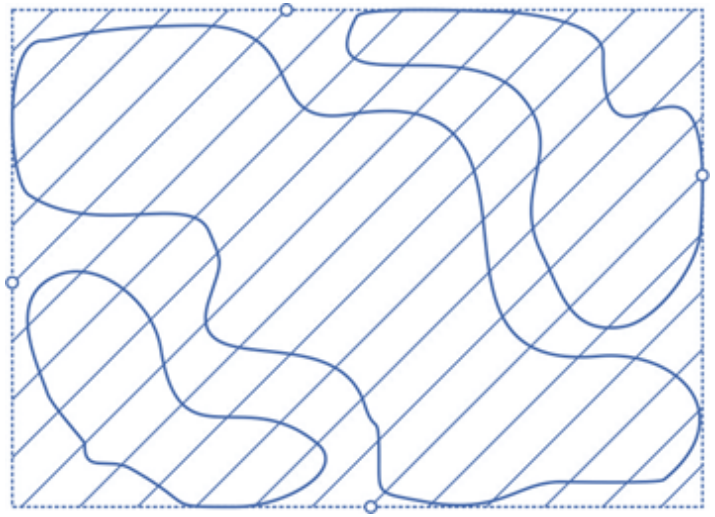
✓ **问题：如何生成能否覆盖整个区域的平行线段序列？**



裁剪填充线

2. 基于裁剪的填充线生成

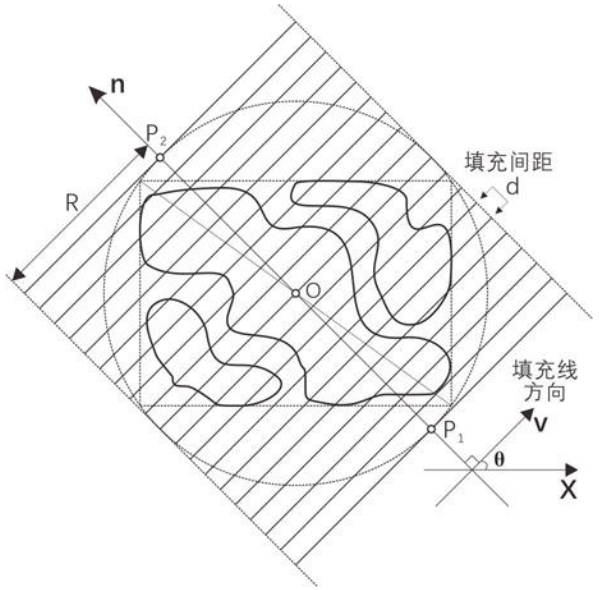
- **平行线段生成原理**
- 理论上，只需在整个平面范围内生成**无数条无限长**的平行线段，那么这些线段必然能够覆盖到整个填充区域。考虑到运算时间和存储空间，这样做不可行。从计算资源优化利用的角度来说，生成的平行线段序列最好能够**恰好**覆盖整个多边形区域。
- **方案一、轴平行包络矩形内生成平行线段**
  - 即便在这种情况下，考虑到线段的长度不定，想要获取这些**线段的端点也并非易事**，需要付出额外的求交运算。
  - **发现**：当平行线倾角变化时，落在包络矩形内部的平行线段的最大长度不会超过包络矩形的对角线长度。因此，理论上，只要设置每条平行线段的长度为包络矩形的对角线长度，就能覆盖整个多边形区域。



在多边形的包括矩形内生成平行线段

2. 基于裁剪的填充线生成

- **平行线段生成原理**
- **方案二、包络矩形外接圆内生成平行线段**
  - 首先，做包络矩形的外接圆，并记其圆心为O，半径为R；
  - 然后，过O点做垂直于填充线方向（图中**v**）的直线P_1P_2，其中P_1、P_2分别为该直线和圆弧的交点。
  - 在线段P_1P_2上根据指定的填充间距（图中d）取点，并以这些点为垂足做P_1P_2的一系列垂线段，使得垂线段的长度为2R，且中点落在P_1P_2上。这些垂线段就是要求的能够覆盖填充区域且带倾角的平行线段。



覆盖多边形区域的平行线段序列计算原理

步骤1. 计算输入多边形序列在X、Y方向的极值xMin、xMax、yMin，yMax；

步骤2. 计算多边形包括矩形外接圆圆心O点坐标和半径R，其中圆心O坐标为:$(\frac{xMin+xMax}{2},\frac{yMin+yMax}{2})$,半径R长度计算公式为：

$$R=\frac{\sqrt{(xMax-xMin)^2+(yMax-yMin)^2}}{2} \quad (9\text{-}1)$$

步骤3. 根据倾角θ计算填充方向向量v和v的左法向量n，其中v向量坐标（二维）为：$(cos(\theta),sin(\theta))$,n向量的坐标为：$(cos\left(\theta+\frac{\pi}{2}\right),sin\left(\theta+\frac{\pi}{2}\right))$.

步骤4. 计算P_1、P_2点坐标，其中：

$$P_1=O-\mathbf{n}\cdot R \quad (9\text{-}2)$$

$$P_2=O+\mathbf{n}\cdot R \quad (9\text{-}3)$$

步骤5. 在直线P_1P_2上计算从P_1开始，间距为d的垂足点序列，其中第i个垂足点（记为Q_i）的计算公式为：

$$Q_i=P_1+\mathbf{n}\cdot(d\cdot i),\ 0\le i\le 2R/d. \quad (9\text{-}4)$$

步骤6. 根据垂足点序列计算平行线段序列，其中第i条平行线段两端点（记为A_i、B_i）的计算公式为：

$$A_i=Q_i-\mathbf{v}\cdot R \quad (9\text{-}5)$$

$$B_i=Q_i+\mathbf{v}\cdot R \quad (9\text{-}6)$$

以上6个步骤的输入为多边形序列、填充间距d以及填充倾角θ，最终输出为能够覆盖整个多边形区域的带倾角的平行线段序列。

2. 基于裁剪的填充线生成

- **填充线裁剪与收集**

- 在GenHatch模块中给出基于Clipper库裁剪功能的区域填充线生成函数**genClipHatches**。

GenHatch.py | **GenHatch模块，genClipHatches函数实现，基于Clipper库的填充线生成** | **9-6**

```
1  def genClipHatches(polygons, interval, angle):                    # 定义基于裁剪的填充线生成函数
2      xMin, xMax = float('inf'), float('-inf')                      # 初始化多边形在X方向极值
3      yMin, yMax = float('inf'), float('-inf')                      # 初始化多边形在Y方向极值
4      z = polygons[0].points[0].z
5      for poly in polygons:                                         # 遍历获取多边形各向极值
6          for pt in poly.points:
7              xMin, xMax = min(xMin, pt.x), max(xMax, pt.x)
8              yMin, yMax = min(yMin, pt.y), max(yMax, pt.y)
9      v = Vector3D(math.cos(angle), math.sin(angle))                # 填充方向向量
10     n = Vector3D(math.cos(angle+math.pi/2), math.sin(angle+math.pi/2))          # 填充方向法向向量
11     O = Point3D((xMin+xMax)/2, (yMin+yMax)/2, z)                  # 多边形包络矩形外接圆中心
12     R = math.sqrt((xMax-xMin)**2 + (yMax-yMin)**2)/2              # 外接圆半径
13     P1 = O - n.amplified(R)                                       # 计算P1点，见图9.9
14     parallels = []                                                # 保存生成的平行线段
15     for i in range(0, int(2*R/interval)+1, 1):
16         Q = P1 + n.amplified(interval*i)                          # 计算P1P2上的垂足点Q
17         seg = Polyline()                                          # 用多段线保存平行线段
18         seg.addPoint(Q - v.amplified(R))                          # 平行线段起点
19         seg.addPoint(Q + v.amplified(R))                          # 平行线段终点
20         parallels.append(seg)
21     hatchSegs = []                                                # 保存最终生成的填充线段
22     ca = ClipperAdaptor()                                         # 创建Clipper的适配器对象ca
23     clipper = Pyclipper()                                         # 创建Clipper对象clipper
24     clipper.AddPaths(ca.toPaths(polygons), PT_CLIP, True)         # 向clipper添加裁剪窗口多边形
25     clipper.AddPaths(ca.toPaths(parallels), PT_SUBJECT, False)          # 向clipper添加平行线段列表
26     sln = clipper.Execute2(CT_INTERSECTION)                       # 执行求交裁剪操作，结果为sln
27     for child in sln.Childs:                                      # 遍历，从sln中提取填充线
28         if len(child.Contour) > 0:
29             poly = ca.toPoly(child.Contour, z, False)             # 将Clipper库Path转化为Polyline
30             seg = Segment(poly.startPoint(), poly.endPoint())     # 将Polyline转化为Segment对象
31             hatchSegs.append(seg)
32     return hatchSegs                                              # 最终返回Segment类型列表
```

获取多边形包络

生成平行线段

Clipper裁剪

2. 基于裁剪的填充线生成

- **填充线裁剪与收集**

- **genClipHatches**函数包含3个输入参数，其中polygons为待填充的多边形区域， interval为填充线间距， angle为填充线倾角。genClipHatches和genSweepHatches两个函数的输入参数完全一致。
- 对裁剪部分代码：
  - 首先创建了一个ClipperAdaptor类的对象ca，作用就是对两种不同格式的多段线进行相互转换。
  - 随后进入裁剪操作主体代码，包括创建Clipper对象clipper，向clipper分别添加裁剪窗口多边形（即输入的polyhons，PT_CLIP）和待裁剪的平行线段（PT_SUBJECT）。
  - 最后调用Clipper库的Execute2函数进行求交操作（CT_INTERSECTION）以获得位于裁剪窗口内部的填充线段。

说明：

面向Python的Clipper库包含两个版本的裁剪执行函数，它们功能相似，主要区别在于对输出结果的组织：

✓ **Execute** 输出的是表示裁剪结果多边形（多段线）的线性列表

✓ **Execute2** 输出的是带有父子节点结构的裁剪结果

对这里需要处理的开放线段裁剪问题，必须调用Execute2函数，否则我们将无法得到裁剪结果

- 在得到Execute2的结果sln后，需要从中将裁剪得到的线段提取出来，方法是遍历sln的子节点（Childs）。每个子节点包含一个Path类型的轮廓（Contour），这些轮廓就是要求的填充线段。需要做的就是将sln的子节点轮廓转化为Polyline对象，然后再将Polyline对象转化为Segment对象并保存到填充线段列表中。

2. 基于裁剪的填充线生成

- **填充线排布探索**
- 使用扫描线法生成的填充线在平面上的分布情况是完全可知、可控的，即根据其生成原理事先预测填充线的排列顺序。
- 但是，由于不了解Clipper内部的裁剪机理，由Clipper生成的填充线在平面上的**分布情况是未知**的。如果不去探索求证，在最坏的情况下，完全有理由假设这些填充线在平面上是随机散乱分布的。
- 对genClipHatches函数进行测试，并在生成的填充线上标记每条线段的序号，以此查看这些填充线段在平面上的分布情况。

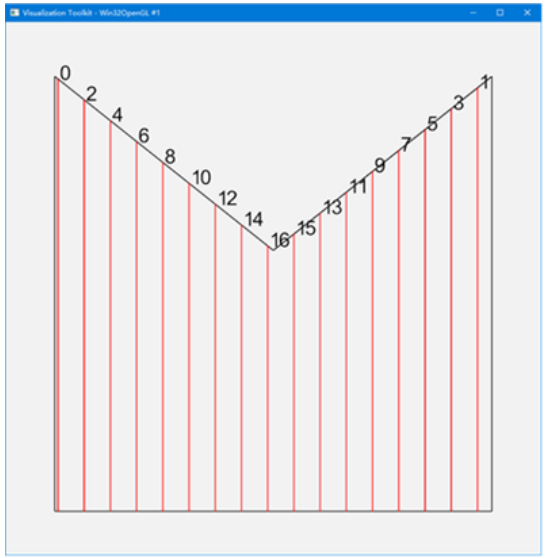
Test_hatch.py	测试基于Clipper库的填充线生成方法及填充线在平面上的分布情况	9-7

```
1   ……                                                  # 导入相关库
2   va = VtkAdaptor()                                   # 创建VTK适配器对象，用于显示
3   path = [(0,0), (100,0), (100,100), (50,60), (0,100)]  # 创建平面点列（多边形）
4   poly = ClipperAdaptor(0).toPoly(path)               # 将平面点列转化为Polyline对象
5   angle = degToRad(90)                                # 填充角度，90度
6   segs = genClipHatches([poly], 6, angle)             # 生成填充线段，间距为6
7   va.drawPolyline(poly).GetProperty().SetColor(0, 0, 0)          # 绘制多边形，黑色
8   for i in range(len(segs)):                          # 遍历填充线
9       va.drawSegment(segs[i]).GetProperty().SetColor(1, 0, 0)    # 绘制填充线段，红色
10      textSrc = vtk.vtkVectorText()                   # 创建VTK文本
11      textSrc.SetText('%d' % i)                       # 设置文本值为当前线段序号
12      textActor = va.drawPdSrc(textSrc)               # 绘制文本
13      textActor.SetPosition(segs[i].B.x, segs[i].B.y, segs[i].B.z)   # 设置文本坐标
14      textActor.SetScale(3)                           # 设置文本字体缩放倍数为3
15      textActor.GetProperty().SetColor(0, 0, 0)       # 设置文本颜色为黑色
16  va.display()
```

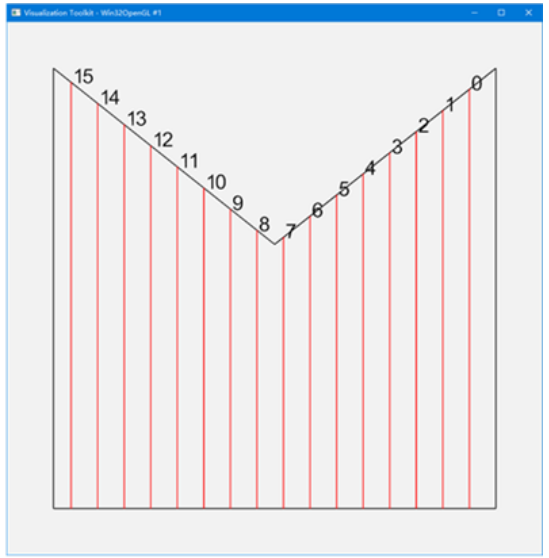
绘制文本

2. 基于裁剪的填充线生成

- **填充线排布探索**
- 由左图可得出以下结论：使用Clipper库裁剪生成的填充线在平面上的排布**不具备连续性**。
  - 理论上，只要有一条填充线段的序号不连续，在对这些线段进行连接的之前，就需要对它们进行重新整理和排布。
  - 如果不重新排布，那么从最左边的第0号线段连到最右边的第1号线段时，将花费大量的空走行程，这在实际三维打印过程中只会增加打印时间，降低打印效率。
- 如果将代码中的genClipHatches替换成**genSweepHatches**，最终可得到如右图所示的填充线及相应序号。使用扫描线法得到的填充线的序号在平面上的排布是连续的。序号连续的好处是可直接根据序号对这些填充线段进行首尾连接，从而降低后续路径连接的难度。



(a) 函数genClipHatches生成的填充线



(b) 函数genSweepHatches生成的填充线

使用两种方法生成的填充线序号分布

2. 基于裁剪的填充线生成

- **填充线排布探索：角度/弧度转换**

- 在代码中，使用了**Utility**模块中的一个函数，**degToRad**，这个函数的作用就是将角度转化为弧度。原则上，它应该还有一个对偶函数，**radToDeg**，作用是将弧度转化为角度。这两个函数虽然简单，但非常通用，因此把它们放在Utility模块中，实现如下：

Utility.py	通用功能模块。添加regToRad和radToDeg函数，角度和弧度相互转换	9-8

```
1  ……                                  # 导入相关库，省略其他代码
2  def degToRad(deg):                  # 角度转化为弧度
3    return deg * math.pi / 180.0
4  def radToDeg(rad):                  # 弧度转化为角度
5    return rad * 180.0 / math.pi
```

下一步工作

目前得到的填充线段是离散的，需要进一步将这些离散填充线段连成一体，生成填充路径。

3. 填充区域分区

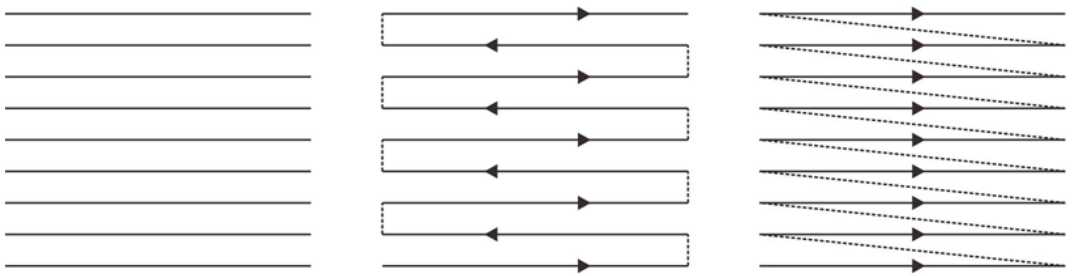
- **填充线连接概述**

- **最简单的填充线连接情形**：针对某一**矩形区域**，假设之前已经用扫描线法在其内部生成了一组平行填充线段，现在要做的就是将这些线段连成一体，使得得到的多段线满足三维打印工艺需求或打印效率需求。一般而言，采用**zig-zag（之字形）**或**单向**这两种形式去有序地连接填充线段。

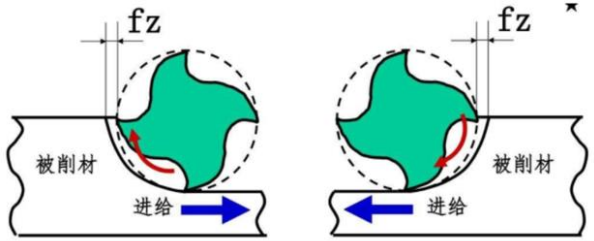
  - 由扫描线法生成的填充线段，每条线段的方向从起点指向终点，所有填充线段的方向都是一致的。
  - **zig-zag连接**的特点是在两条相邻平行线段的同侧点（同为起点或同为终点）处连接这两条线段。
  - **单向连接**的特点是在两条相邻平行线段的异侧点（一个为起点、一个为终点）处连接这两条线段。
  - 相比较而言，zig-zag路径长度较短，加工效率较高；而单向路径长度较长，加工效率较低。

- 在数控加工中，由于刀具进给和主轴旋转的共同作用，常规的铣削加工工艺一般可分为**顺铣**和**逆铣**。

  - zig-zag路径既包含顺铣过程，又包含逆铣过程；而由单向路径只包含其中一种铣削过程。
  - 对机加工路径规划而言，对那些表面制造精度要求较高的零件，一般会采用单向连接策略。但三维打印工艺不存在顺铣和逆铣的区别，因此，为保证打印效率，在三维打印中，对平行填充线，一般会选用zig-zag这种路径连接方式。



平行填充线段连接方式



逆铣
进给方向和旋转方向相反
从下往上刨

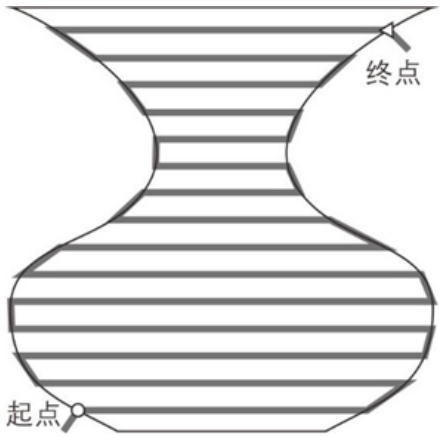
顺铣
进给方向和旋转方向相同
从上往下刨

逆铣：
- 刀具使用寿命低
- 加工表面质量差
- 可铣带硬皮的工件
- 工作台不会窜动

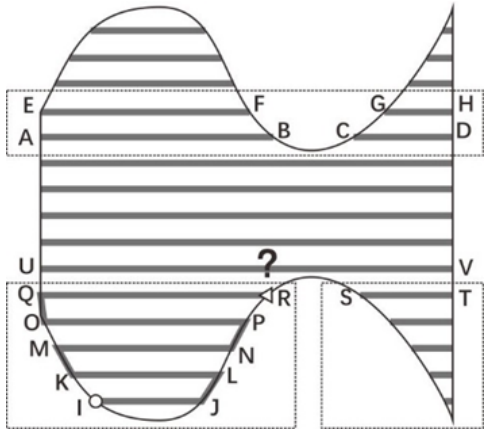
顺铣：
- 刀具使用寿命高
- 加工表面质量好
- 不可铣带硬皮的工件
- 工作台可能会窜动

3. 填充区域分区

- **填充线连接概述**
- 对左图花瓶形状的多边形，当填充角度为**0度**时，可以轻松地将所有填充线段连成一体。
- 如果将花瓶顺时针旋转**90度**（右图），而保持填充角度为0度不变，此时无法将所有填充线段连成一体。在连接这些线段时，将遭遇两个问题。
- **问题一、上下填充线段的识别配对**
  - 如果调用**genSweepHatches**生成填充线段，如右图，上方局部区域内的4条填充线段在函数返回结果中的排列顺序是：AB、CD、EF、GH。
  - 假如在某一时刻，已经通过zig-zag方式连接了AB线段，现在要从B点连接到下一条填充线段。由图可知，从zig-zag连接逻辑上来说，AB的下一条线段应该是EF；这和填充线段列表中线段的顺序是违背的，因为在列表中，AB的下一条线段是CD。
  - 因此有必要根据当前线段AB从填充线段列表中搜索并识别符合zig-zag连接逻辑的下一条线段。



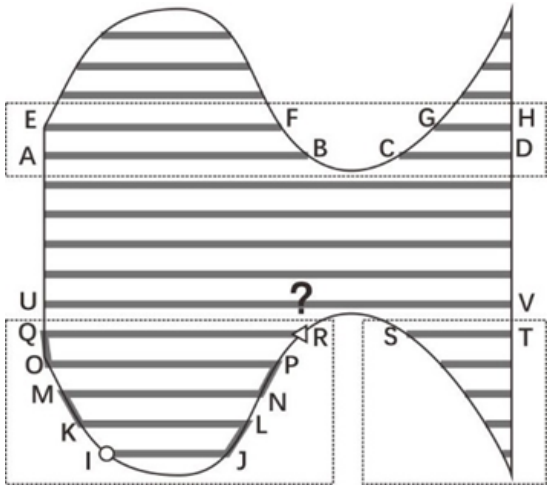
zig-zag连成一体的填充线



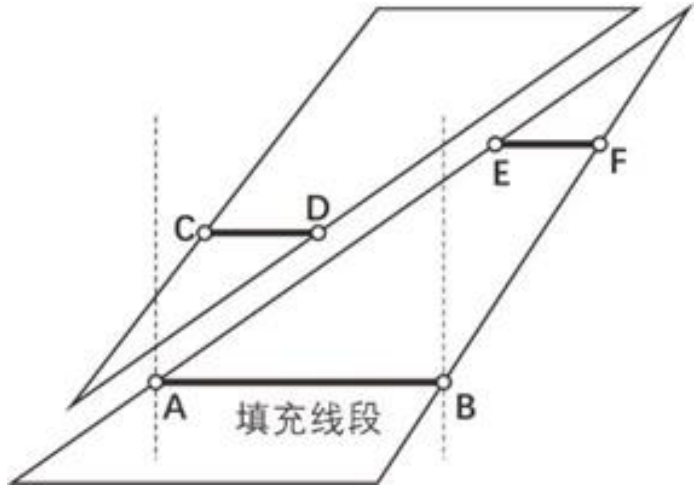
无法连成一体的填充线

3. 填充区域分区

- **填充线连接概述**
- 产生**问题一**的**根本原因**是上下层扫描高度上都存在**至少大于一条**填充线段。
- 虽然看起来简单，但解决这个问题时却极容易出错，特别是当多边形轮廓形状比较复杂时，很难制定正确、完备的识别标准。
  - **一个识别标准**是下一条线段所在的直线和当前线段所在的直线的**间距是给定的填充间距值**——但这只是其中一个基础条件。
  - **另一个识别标准**可能是下一条线段的投影落在当前线段上。但这个条件**不一定正确**，如右图所示，线段CD的投影落在AB上，线段EF的投影没有落在AB上，但EF却是AB的下一条线段。



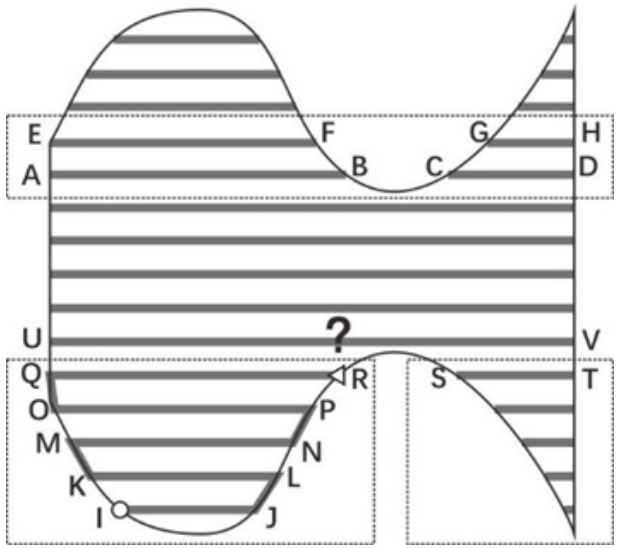
无法连成一体的填充线



投影配对上下层线段失败案例

3. 填充区域分区

- **填充线连接概述**
- **问题二、和当前填充线段配对连接的线段可能根本就不存在**
  - 对图中左下方局部区域内的填充线段，从I点连入，可以轻松地将IJ、KL、MN、OP、QR这5条线段以zig-zag的方式串在一起。
  - 当连到R点时，陷入僵局：QR的下一条线段是哪个？从表面上看，QR的下一条线段应该是UV。如果去连右下方局部区域内的填充线段，UV同时可能又是ST的下一条线段。一条线段同时和两条线段配对，肯定是错的！
  - 在这种情况下，**定义QR和ST的下一条线段都不存在**，即对**局部区域内填充线段的连接到此为止**。这个定义的后果是无法将整个填充区域内的所有线段连成一体，但除此之外，没有更好的办法。



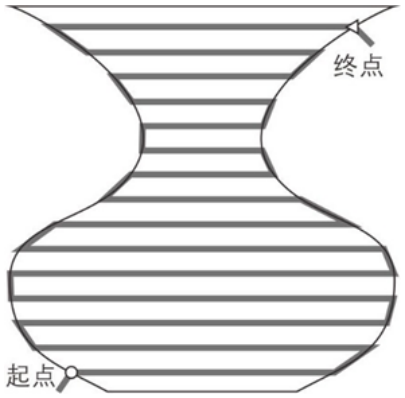
无法连成一体的填充线

3. 填充区域分区

- **填充线连接概述**



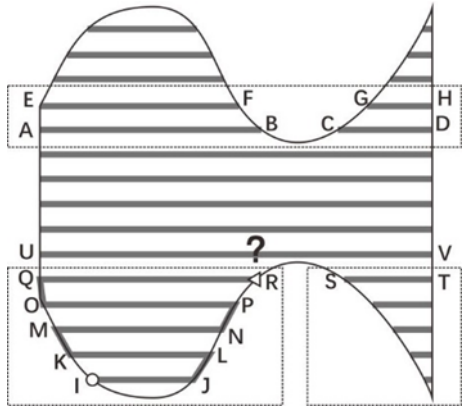
- **思考：对形状完全相同的两个多边形区域，一个填充线可以连成一体，另一个填充线却无法连成一体，这是为什么?**
  - 观察发现：在左图中，**一条扫描线上只存在一条填充线段**；而在右图中，**一条扫描线上可能存在若干条填充线段**。对前一种情况，可以形象地称扫描线被多边形轮廓**切割成了一段**；对后一种情况，称扫描线被多边形轮廓**切割成了多段**。
  - 不难想象，在右图，如果把位于同一扫描线上的线段间的缝隙（位于区域外部）**补全**，那么照样可以轻松地将所有线段以zig-zag的形式连成一体。
- **结论：一个封闭区域内部的所有填充线能否连成一体和该区域内扫描线被切割得到的段数有关。**
  - 定义：如果区域内所有扫描线均被切割成一段，那么得到的所有填充线段可以简单地以zig-zag的形式被连成一体，称这种区域为**“单连通”区域**，对应的多边形为**“单联通”多边形**；
  - 定义：如果这个区域内所有扫描线中存在一些扫描线被切割成多段，那么得到的填充线段将无法被连成一体，称这种区域为**“多连通”区域**，对应的多边形为**“多连通”多边形**。



zig-zag连成一体的填充线

单联通多边形

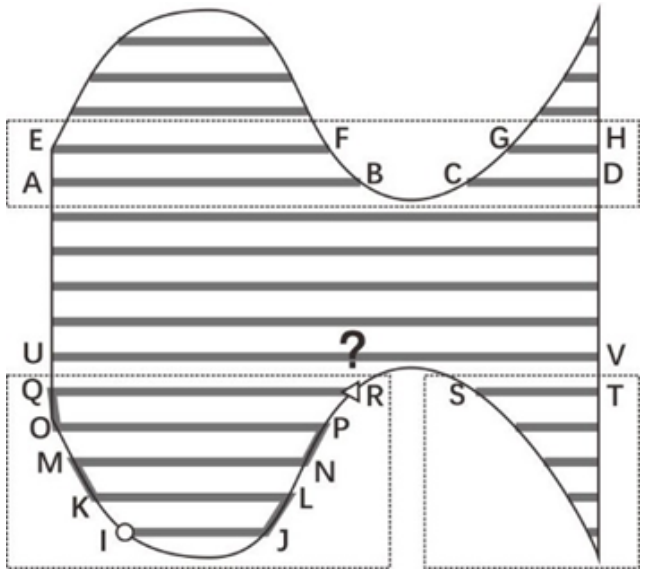
多联通多边形



无法连成一体的填充线

3. 填充区域分区

- **填充线连接概述**
- **一种策略：**在生成填充线之前，将输入的多边形区域进行**分区**，使得得到的每个独立的子区域为**单连通区域**。
  - 如果这个策略可行，就可以顺理成章的解决上下层填充线段的识别配对难题（即第一个问题）。
  - 而对每个子区域内生成的连接路径，可以站在一个更为广阔的全局的角度对它们进行优化连接。



3. 填充区域分区

- **填充区域分区原理**

- **联通区域定义：**如果一个多边形区域在每个**扫描**高度上都只有一条填充线段，那么称这个区域为单连通区域；如果一个多边形区域在某个扫描高度上存在多条填充线段，那么称这个区域为多连通区域。

- 根据以上定义，单连通区域和多连通区域都是相对填充方向（即扫描线方向）而言的，如果改变填充方向，同一多边形区域可能在单连通区域和多连通区域间相互切换。

- **目标：**在**指定填充方向**的前提下，对输入多边形区域进行分区，使得得到的所有子多边形相对于填充方向都为单连通区域。

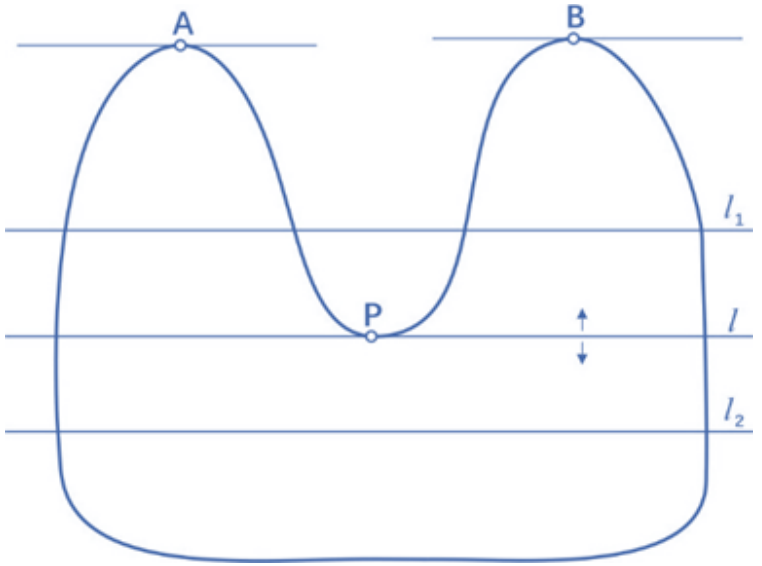
问题

以上关于单联通/多连同区域的定义是以结果为导向的，不可能先生成填充线，再去根据填充线数量进行分区，因此以上定义无法真正指导如何对多边形分区，实现以上目标！

3. 填充区域分区

- **填充区域分区原理**
- 定义区域连通特性涉及一个关键指标，即扫描线上线段的数量，或者说是扫描线被区域轮廓**切割**得到的线段数量。下面从动态平移扫描线的视角来看一下“**切割**”问题。

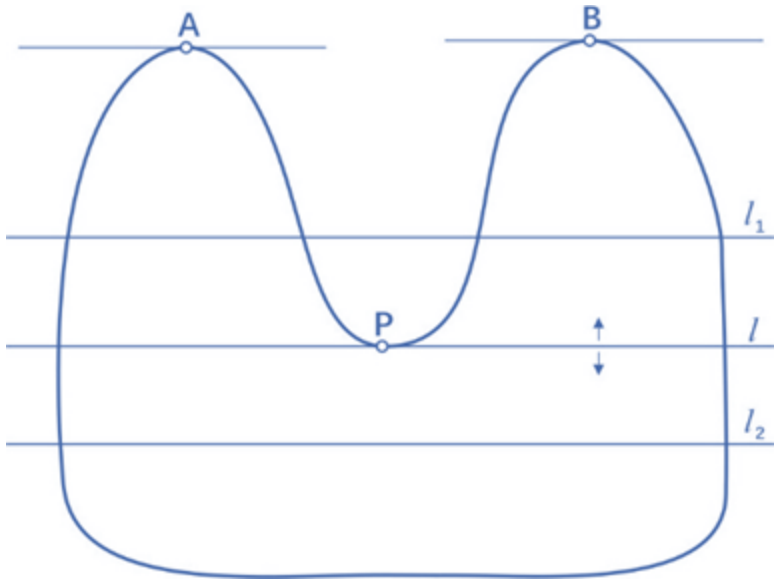
- 先降低难度，假设填充线倾角为0，对图中简单凹多边形多连通区域：
  - 在直线l_2位置时，扫描线和区域轮廓有2个交点，被轮廓切割得到一段填充线。
  - 在直线l_1位置时，扫描线和区域轮廓有4个交点，被轮廓切割得到两段填充线。
  - 在扫描线从l_2位置逐渐上移到l_1位置的过程中，可以预测，扫描线将在某一时刻和轮廓凹陷处开始相交，在临界状态下，扫描线和区域轮廓有3个交点。
  - 记临界位置为直线l，当扫描线在l位置时，**恰好可以将原多边形区域切分成3个子区域**；而且，在这3个子区域内，在每个扫描高度上，都只有一条填充线段，因此这**3个子区域都是单连通区域。**
- 对如图所示的多连通区域，临界直线l可以将其切分为3个单连通区域，称直线l为**切分直线**。



扫描线位置和区域切分关系

3. 填充区域分区

- **填充区域分区原理**
- 如果将位于直线l上的扫描线向上平移（如移动到l_1位置），此时扫描线仍可将原区域切分为3个子区域，其中上方2个子区域为单连通区域，但下方一个子区域为多连通区域。
- 反之，如果将位于直线l上的扫描线向下平移（如移动到l_2位置），此时扫描线将原区域切分成2个子区域，其中上方子区域为多连通区域，下方子区域为单连通区域。
- **小结：切分直线l的位置并非偶然**，随意对其进行向上或向下平移后，都将无法得到全是单连通区域的子区域。



切分直线

扫描线位置和区域切分关系

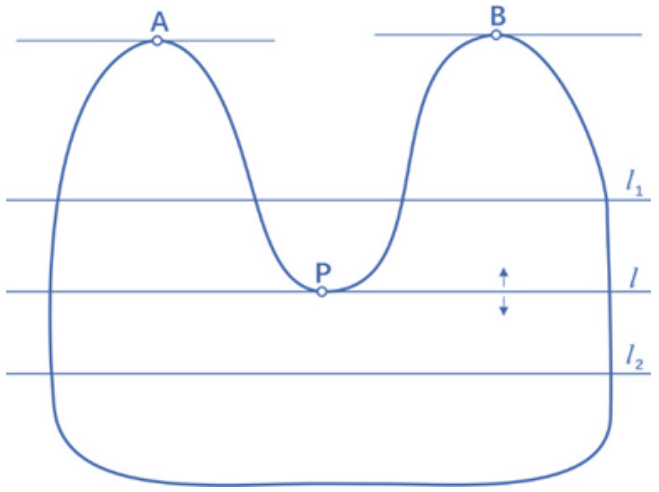
3. 填充区域分区

- **填充区域分区原理**

进一步观察：在临界直线*l*上时，扫描线和区域轮廓的**交点个数恰好发生了变化**，即此时扫描线和区域轮廓之间多了（或少了，从高往低扫描时）一个交点，不妨记这个交点为P。

- **如何确定临界直线 *l* 的位置?**
  - 理论上，可将扫描高度间距设置为一个趋向0的正实数，从下至上遍历这些扫描高度时，必然能够找出扫描线和区域轮廓交点数量恰好发生变化的那个扫描高度，进而确定交点P以及直线*l*位置。 ——但该方案显然不具备实际可操作性
- **第一个重要发现**：交点P是区域轮廓的一个**局部峰值点**。理论上，在扫描线移动过程中，必然首先会和轮廓的峰值点相交。

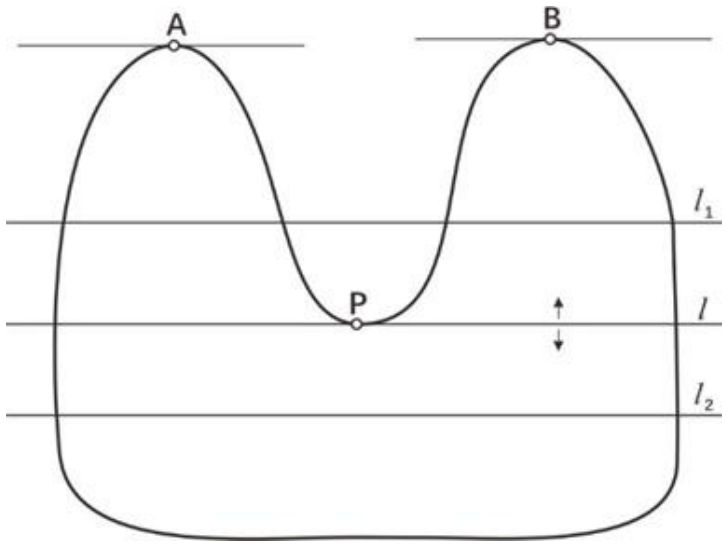
- **小结**：能够将输入多连通区域分割成若干单连通区域的切分直线必然**经过轮廓的峰值点**。



扫描线位置和区域切分关系

3. 填充区域分区

- **填充区域分区原理**
- **逆命题是否成立：**是否所有经过轮廓峰值点的直线都可以将多联通区域切分为单连通区域呢？
  - 对图中的3个峰值点A、B、P，过A点和B点的直线并不能将区域切分成单连通区域，只有过P点的直线才能对区域进行有效切分。
  - 进一步地，A点和B点是局部凸起轮廓上的点，而P点是局部**凹陷**轮廓上的点。
- **第二个重要发现：**只有经过轮廓**凹峰点**的直线才可以将多连通区域切分成若干单连通区域。
- **小结：**要确定切分直线的位置，必须首先找出区域轮廓上的凹点，然后从这些凹点中找出凹峰点。



扫描线位置和区域切分关系

3. 填充区域分区

- **填充区域分区原理：凹点寻找方法**

- 对于平面上一个由一组简单多边形围成的任意平面区域，如果各多边形的方向符合“**外逆内顺**”的规定，那么可以使用**向量叉乘**法确定轮廓上的某一点P_k是否为凹点，步骤如下：

步骤1. 计算向量$\mathbf{v}_k$、$\mathbf{v}_{k+1}$，使得：

$$\mathbf{v}_k = P_k - P_{k-1}, \tag{9-7}$$

$$\mathbf{v}_{k+1} = P_{k+1} - P_k, \tag{9-8}$$

式中P_{k-1}和P_k分别为轮廓上P_k的上一点和下一点。

步骤2. 如果P_k满足：

$$(\mathbf{v}_k \times \mathbf{v}_{k+1}).\mathrm{dz} < 0, \tag{9-9}$$

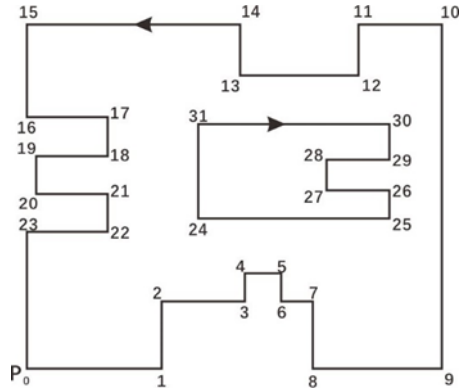
则P_k为凹点，式中‘×’为两向量叉乘符号；如果：

$$(\mathbf{v}_k \times \mathbf{v}_{k+1}).\mathrm{dz} > 0, \tag{9-10}$$

则P_k为凸点；如果：

$$(\mathbf{v}_k \times \mathbf{v}_{k+1}).\mathrm{dz} = 0, \tag{9-11}$$

则P_k为平点，即$P_{k-1}P_k$和P_kP_{k+1}平行。



根据以上步骤，得出：

- 在图中外边界轮廓上，点P_0、P_1、P_8、P_9、P_{10}、P_{11}、P_{14}、P_{15}、P_{16}、P_{23}等为凸点；而点P_2、P_7、P_{12}、P_{13}、P_{17}、P_{18}、P_{21}、P_{22}等为凹点；
- 而在内边界轮廓上，点P_{27}、P_{28}为凸点，点P_{24}、P_{25}、P_{26}、P_{29}、P_{30}、P_{31}等为凹点。
- 对于具有凹凸嵌套关系的局部轮廓，如P_4、P_5、P_{19}、P_{20}，通过以上步骤，也能识别出其中凹点或凸点。

- **轮廓上点的凹凸属性具有旋转不变性，即和轮廓在平面上的摆放角度无关。**
- **即便对输入轮廓进行旋转操作，轮廓上原来的凹点仍为凹点，原来的凸点仍为凸点。**

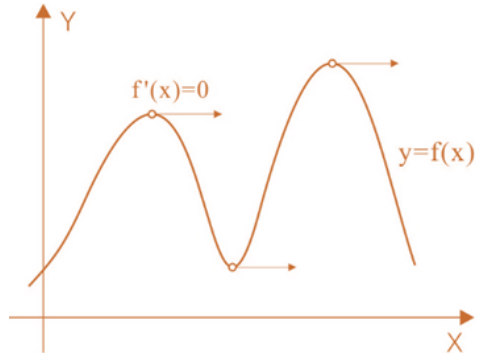
3. 填充区域分区

- **填充区域分区原理：峰值点寻找方法**

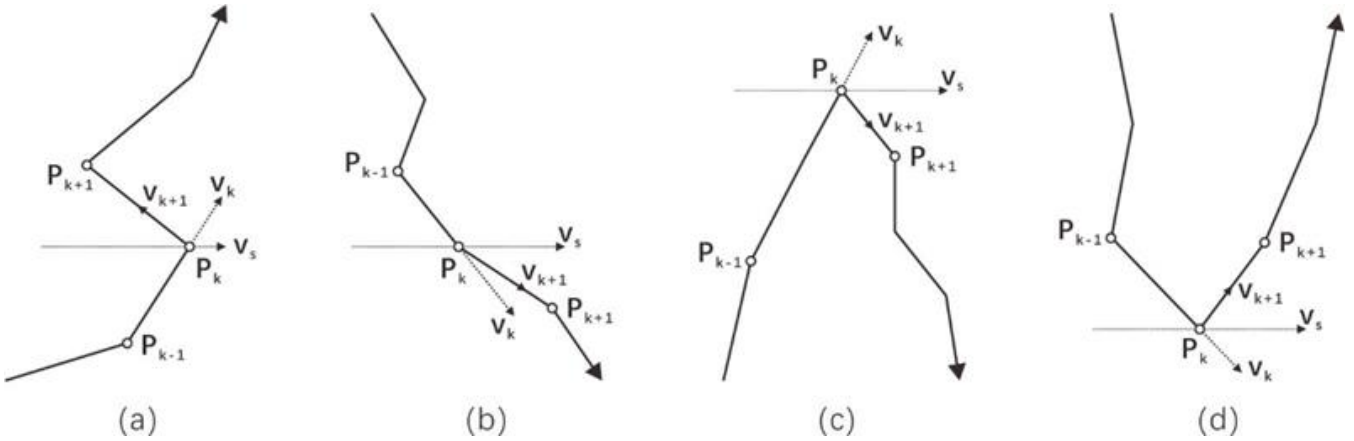
- 对连续可导的函数 $y = f(x)$，在它的函数图像上，峰值点在$f'(x) = 0$（即导数为0）时取到。
  - 这里涉及的多边形轮廓是离散的顶点序列，非连续且不可导，因此不能用以上代数法求取轮廓上的峰值点。

- 对区域轮廓上的某一顶点P_k，向量$\mathbf{v}_k$、$\mathbf{v}_{k+1}$由公式(9-7)和(9-8)计算得到，向量$\mathbf{v}_s$为扫描线方向向量：
  - 如果局部轮廓是单调上升或单调下降的，如右图a和b所示，那么向量$\mathbf{v}_k$、$\mathbf{v}_{k+1}$将指向$\mathbf{v}_s$的同侧；
  - 如果局部轮廓在P_k点转折，达到峰值，如右图c和d所示，那么向量$\mathbf{v}_k$、$\mathbf{v}_{k+1}$将指向$\mathbf{v}_s$的异侧。

- **结论：向量$\mathbf{v}_k$、$\mathbf{v}_{k+1}$指向$\mathbf{v}_s$的异侧是点P_k为轮廓峰值点的充要条件。**



连续可导函数峰值确定方法



离散多边形轮廓峰值确定方法

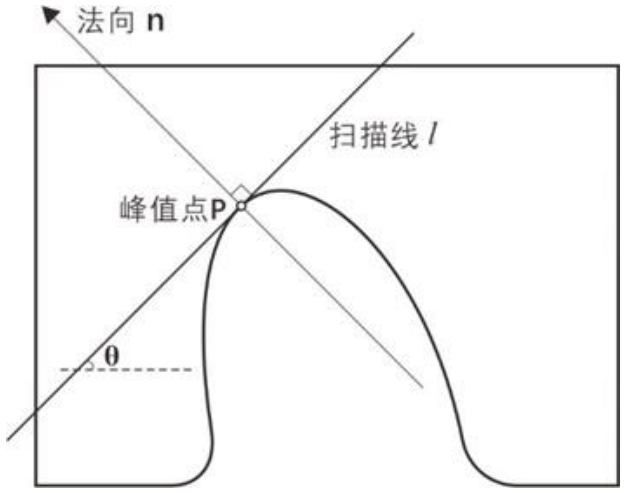
可用**向量叉乘法**判断两个任意向量$\mathbf{v}_k$和$\mathbf{v}_{k+1}$分别指向给定向量$\mathbf{v}_s$的异侧。如果$\mathbf{v}_k \times \mathbf{v}_s$得到的向量和$\mathbf{v}_{k+1} \times \mathbf{v}_s$得到的向量所指的方向相反，更具体的，只需满足：

$$(\mathbf{v}_k \times \mathbf{v}_s).\mathrm{dz} \cdot (\mathbf{v}_{k+1} \times \mathbf{v}_s).\mathrm{dz} \le 0 \qquad (9\text{-}12)$$

就可以判断向量$\mathbf{v}_k$、$\mathbf{v}_{k+1}$指向$\mathbf{v}_s$异侧。式中取到等号是因为当$\mathbf{v}_k$或$\mathbf{v}_{k+1}$和$\mathbf{v}_s$平行时，P_k也为峰值点。

3. 填充区域分区

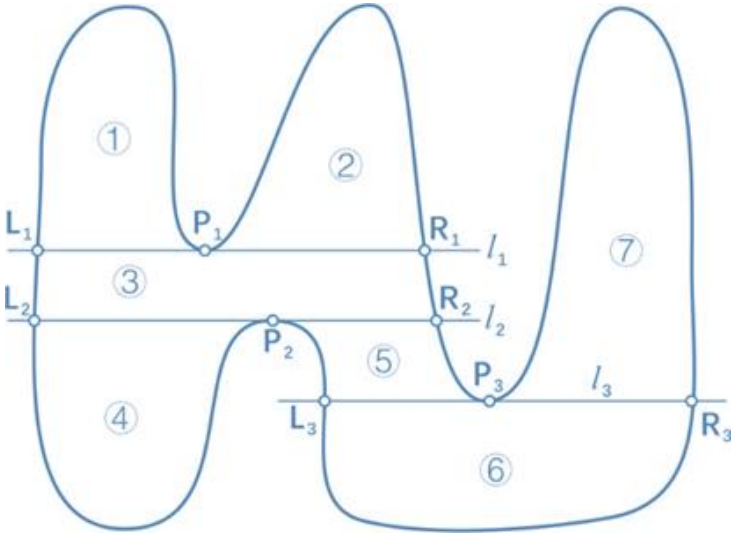
- **填充区域分区原理：凹峰点寻找方法**
- 当扫描线倾角为0时，对区域轮廓上的任意一点P_k，如果它**同时满足式(9-9)和式(9-12)**，那么这个点就是凹峰点。
- 当扫描线倾角不为0时，如何寻找凹峰点？
  - 轮廓上点的凹凸属性具有旋转不变性，因此当扫描线倾角非0时，寻找凹点的方法（即式(9-9)）仍然有效。
  - 轮廓的峰值是一个相对的概念，它是相对于扫描线的法向上的极值。当扫描线倾角为0时，很容易理解轮廓上的局部最高点或最低点就是峰值；但当扫描线倾角非0时，局部最高点和最低点应从扫描线的法线方向考虑和计算。从这个角度来说，寻找峰值点的方法（即式(9-12)）也同样适用于扫描线倾角非0的情况。
- **结论：在填充倾角非0的情况下，对区域轮廓上的任意一点P_k，如果它同时满足式(9-9)和式(9-12)，则P_k为凹峰点。**



扫描线倾角非0时局部轮廓上的峰值点

3. 填充区域分区

- **填充区域分区原理：根据凹峰点分区**
- 如图所示，对给定的多连通区域和填充方向，可以找出3个凹峰点，分别记为P_1、P_2、P_3，并记l_1、l_2、l_3为分别过这三个点的切分直线。图中过每个凹峰点的**切分直线（线段）不能太长**，否则会对原多连通区域带来一些不必要的分区。
  - 对点P_1，从l_1和区域轮廓的所有交点中找出在P_1两侧且距离P_1最近的两个点，记这两点分别为L_1和R_1，根据切分线段L_1R_1可以将原多连通区域划分成3个子区域。其中2个位于上方的子区域（区域1和2）为单连通区域，1个位于下方的子区域仍然为多连通区域。
  - 同理，对得到的新多连通区域，根据过P_2的线段L_2R_2对其进一步分区，此时又可得到2个单连通区域（区域3和4）和1个多连通区域。
  - 最后，对剩下的多连通区域，根据过P_3的线段L_3R_3对其进一步分区，此时可以得到3个单连通区域（区域5、6、7）。
- 对输入的原始多连通区域，通过3次分区，可以得到7个单连通区域。



根据凹峰点切分多连通区域方法

凹峰点数量决定了分区次数，而分区次数又决定了单连通区域个数，对任意多连通区域，进一步归纳和总结，最终可得到如下关系式：

$$N_s = 2N_p + 1, \qquad (9\text{-}13)$$

其中N_s为最终生成的单连通区域个数，N_p为输入区域轮廓上的凹峰点个数。

3. 填充区域分区

- **填充区域分区实现：总体步骤**
- 根据以上原理，将输入多连同区域切分成若干单连通区域的步骤如下：
  - **步骤1.** 输入多边形区域，每个多边形的方向需符合“**外逆内顺**”的规定。
  - **步骤2.** 根据式(9-9)和(9-12)搜索并存储多边形顶点中的所有凹峰点。
  - **步骤3.** 对每个凹峰点，构造一条经过它的**切分线段**。
  - **步骤4.** 沿切分线段将多边形分割成若干单连通子区域。
  - **步骤5.** 输出得到的单连通子区域序列。
- **说明：**以上步骤没有提及填充倾角，这是因为对输入区域进行分区是整个平行填充路径生成过程的其中一部分，这部分工作在对区域轮廓旋转之后，因此，认为在对填充区域进行分区时，填充线的倾角为0，即在以上步骤中，扫描线倾角为0。

3. 填充区域分区

- **步骤1.** 输入多边形区域，每个多边形的方向需符合“外逆内顺”的规定。
- **步骤2.** 根据式(9-9)和(9-12)搜索并存储多边形顶点中的所有凹峰点。

• 填充区域分区实现：步骤1、2实现

- 步骤1中，如果输入的多边形方向不满足“外逆内顺”的要求，有必要对其进行调整。以上两个步骤的对应实现代码如下：对步骤2，因为之前已经详细介绍了凹峰点的计算原理和方法，并给出了详细公式，因此，这一步也非常明确。将和平面区域分区相关的代码写在SplitRegion类中，

SplitRegion.py | **区域分区模块，部分代码，输入及凹峰点寻找函数** | **9-9**

```
class SplitRegion:
    def __init__(self, polygons, adjustPolyDirs = False):          # 初始化函数
        self.polygons = polygons                                    # 在类中保存多边形
        if adjustPolyDirs:                                          # 如果需要调整多边形方向，则
            adjustPolygonDirs(self.polygons)                        # 将其调整成外逆内顺的方向
        self.splitPolygons = self.split()                           # 分区结果存储在splitPolygons中
    def split(self):
        pass                                                        # 略过split函数实现
    def findTurnPoints(self):                                       # 凹峰点寻找函数
        vx = Vector3D(1, 0, 0)                                      # 定义和X轴平行的向量vx
        turnPts = []                                                # 定义凹峰点列表turnPts
        for poly in self.polygons:                                  # 遍历多边形列表
            for i in range(poly.count()-1):                         # 遍历多边形中的每个点
                pts = poly.points
                v1 = pts[-2 if (i==0) else (i-1)].pointTo(pts[i])  # 前一点到当前点向量
                v2 = pts[i].pointTo(pts[i+1])                       # 当前点到下一点向量
                if v1.crossProduct(vx).dz * v2.crossProduct(vx).dz <= 0:                    # 满足式(9-12)，峰点
                    if v1.crossProduct(v2).dz < 0:                                          # 是否满足式(9-9)，凹点
                        turnPts.append(pts[i])                      # 同时满足，则添加至凹峰点列表
        return turnPts                                              # 返回凹峰点列表
```

3. 填充区域分区

- 步骤3. 对每个凹峰点，构造一条经过它的切分线段。

• 填充区域分区实现：步骤3实现（构造切分线段）

- 步骤3的核心是计算过凹峰点的扫描线和多边形的所有交点，然后在从所有交点中筛选距离凹峰点**两侧最近**的两个交点。这一步事实上需要处理两个问题：一是求交，二是筛选。
  - **求交问题**。考虑到可能需要同时处理多个凹峰点，即解决多条扫描线和多边形的求交问题，这个问题和填充线交点计算问题十分相似。进一步比较，发现**calcHatchPoints**函数完全适用于这里的求交场景。该函数的输入包含多边形列表和扫描高度列表两个部分，对这里的求交场景，多边形列表就是需要分区的多边形区域，扫描高度列表就是一系列由凹峰点y值组成的列表。
  - **筛选问题**。**calcHatchPoints**函数返回的是交点的二维列表，其中第一个维度表示不同层高，第二个维度表示同一层高上不同的交点。以某个凹峰点为例，要从以上二维交点列表中找出同一高度上距离该凹峰点两侧最近的两个交点。
- 为了实现筛选目标，在SplitRegion类中定义一个**findLRPoints**函数（L、R分别表示left和right），具体实现代码如下：

SplitRegion.py	区域分区模块，部分代码，实现findLRPoints和createSplitter两个函数	9-10

```
1  class SplitRegion:
2      ……                                  #        忽略类中其他代码
3      def findLRPoints(self, pt, ptses):  # 寻找凹峰点两侧交点函数
4          for pts in ptses:               # 遍历二维点列表第一个维度
5              if len(pts) > 0 and pts[0].y == pt.y:   # 定位高度相等的层
6                  for i in range(len(pts)-1):         # 遍历列表第二个维度
7                      if pt.x > pts[i].x and pt.x < pts[i+1].x:   # 通过x坐标定位凹峰点两侧点
8                          return pts[i], pts[i+1]     # 返回找到的两侧点
9          return None, None               # 如果没找到，则返回None
```

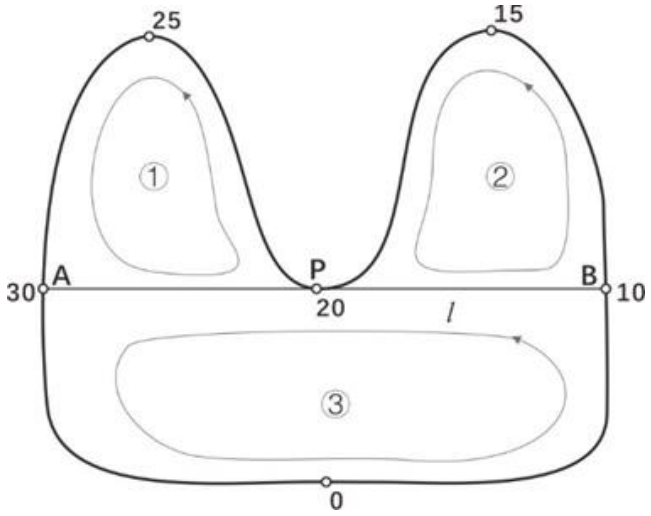
- 在**findLRPoints**函数中，首先从二维列表中找出和给定凹峰点高度（即y值）相同的点列pts，然后再逐一判断凹峰点的x坐标是否落在点列中的相邻点之间，如果落在某相邻点之间，那么这两个相邻点就是凹峰点两侧的最近点。这是为什么呢？因为在calcHatchPoints函数的实现代码中，已经对同一层高上的所有交点按照它们的x坐标值从小到大进行了排序。

3. 填充区域分区

- **步骤4. 沿切分线段将多边形分割成若干单连通子区域。**

- **填充区域分区实现：步骤4实现——最难的一步**

- **常规思路：**针对简单情况，如图，通过一条切分线段将给定平面区域分割成三个部分，生成3条全新的封闭轮廓。
  - 如图，对要分割区域，首先需要计算切分线段和区域轮廓的**交点**，这点已做到。此外，还需要知道交点所在的轮廓边（或顶点）在轮廓上的**序号**，只有这样，才能知道从哪里截取原轮廓中的部分轮廓和切分线段组成新的封闭轮廓。
  - 要识别交点序号非常繁琐，因为**calcHatchPoints**函数在计算交点时并没有保存相交线段的序号信息，因此需要根据交点坐标遍历区域轮廓上的所有边，从中找出各交点所在轮廓边（或顶点）的序号。
  - 如图，不妨设A、P、B三个交点在原轮廓上的序号分别为10、20和30，那么最终生成的3个新的封闭轮廓的部分点序将是：0-10-30-0、10-15-20-10、20-25-30-20。
  - 对复杂的情况，当有多条切分线段时，可按照以上方法逐条递归分割原始区域。当然，在实现递归算法时我们还需要解决诸如当前切分线段属于上一步分区之后的哪个子区域等细节问题，非常繁琐。



根据交点序号建立分区轮廓

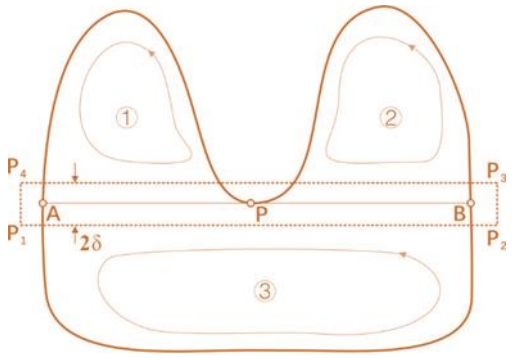
3. 填充区域分区

- 步骤4. 沿切分线段将多边形分割成若干单连通子区域。

- **填充区域分区实现：步骤4实现——最难的一步，有无简单方法?**

- **如图，将切分线段AB替换成切分矩形，**在这种情况下，只需对**输入区域**和**切分矩形**做**布尔差运算**，就能将输入区域分割成3个子区域。

- 切分矩形必须足够**“瘦”**，才能让其逼近切分线段。如图，不妨设参数δ是一个趋向于0的极小正实数，$\mathbf{v}_x$和$\mathbf{v}_y$分别为坐标系X、Y轴的单位方向方向，可构造以如下4个点为顶点的矩形：



使用矩形裁剪重建分区轮廓

$$P_1 = A - \delta \cdot \mathbf{v}_x - \delta \cdot \mathbf{v}_y, \qquad (9\text{-}14)$$

$$P_2 = B + \delta \cdot \mathbf{v}_x - \delta \cdot \mathbf{v}_y, \qquad (9\text{-}15)$$

$$P_3 = B + \delta \cdot \mathbf{v}_x + \delta \cdot \mathbf{v}_y, \qquad (9\text{-}16)$$

$$P_4 = A - \delta \cdot \mathbf{v}_x + \delta \cdot \mathbf{v}_y. \qquad (9\text{-}17)$$

- 因为δ趋向于0，因此矩形在沿X方向上的长度和线段AB相当，而在沿Y方向上的高度为2δ，仍为一个趋向0的极小值，因此，有理由相信，使用矩形$P_1P_2P_3P_4$和使用线段AB对输入区域进行切分的效果等价。

- 对复杂的情况，当有多条切分线段时，仍然可以构造多个切分矩形进行布尔运算。

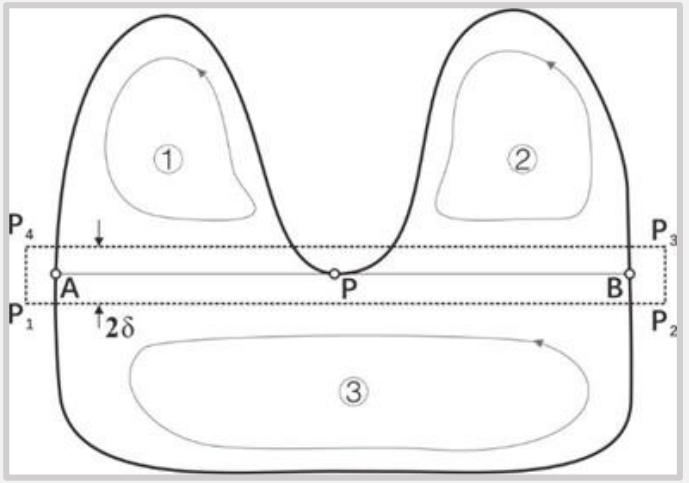
3. 填充区域分区

• 填充区域分区实现：剩余代码

- 下面给出SplitRegion模块的剩余代码，主要包括两个函数，一个是**createSplitter**，另一个是**split**：

SplitRegion.py　区域分区模块，部分代码，实现createSplitter和split两个函数　9-11

```
class SplitRegion:
    def createSplitter(self, p1, p2, delta = 1.0e-4):                    # 创建切分矩形
        vx, vy = Vector3D(1, 0, 0), Vector3D(0, 1, 0)                    # 创建沿X、Y轴的单位向量
        splitter = Polyline()                                            # 切分矩形实际为封闭多段线
        splitter.addPoint(p1 - vx.amplified(delta) - vy.amplified(delta))
        splitter.addPoint(p2 + vx.amplified(delta) - vy.amplified(delta))
        splitter.addPoint(p2 + vx.amplified(delta) + vy.amplified(delta))
        splitter.addPoint(p1 - vx.amplified(delta) + vy.amplified(delta))
        splitter.addPoint(splitter.startPoint())                         # 封闭多段线
        return splitter
    def split(self):                                                     # 定义核心切分流程控制函数
        turnPts = self.findTurnPoints()                                  # 获取轮廓上的所有凹峰点
        if len(turnPts) != 0:                                            # 如果凹峰点个数不为0，则继续
            ys = []
            for pt in turnPts:                                           # 收集扫描高度，并对其排序
                ys.append(pt.y)
            ys.sort()
            hatchPtses = calcHatchPoints(self.polygons, ys)                      # 计算扫描线和轮廓交点
            splitters = []                                               # 定义切分矩形列表
            for turnPt in turnPts:                                       # 根据凹峰点构造切分矩形
                lPt, rPt = self.findLRPoints(turnPt, hatchPtses)                 # 寻找凹峰点两侧最近点
                if lPt is not None and rPt is not None:
                    splitter = self.createSplitter(lPt, rPt)             # 根据两侧最近点构造切分矩形
                    splitters.append(splitter)
            if len(splitters) != 0:                                      # 如果切分矩形个数不为0，继续
                clipper, ca = Pyclipper(), ClipperAdaptor()              # 构造Clipper对象
                clipper.AddPaths(ca.toPaths(self.polygons), PT_SUBJECT)
                clipper.AddPaths(ca.toPaths(splitters), PT_CLIP)
                sln = clipper.Execute(CT_DIFFERENCE)                     # 布尔差运算
                return ca.toPolys(sln, turnPts[0].z)                     # 返回裁剪结果
        return self.polygons                                             # 对单连通区域，则直接返回输入
def splitRegion(polygons, adjustPolyDirs = False):                       # 定义SplitRegion类的接口函数
    return SplitRegion(polygons, adjustPolyDirs).splitPolygons
```



使用矩形裁剪重建分区轮廓

3. 填充区域分区

- **填充区域分区实现：剩余代码**
- **createSplitter**函数的输入包括凹峰点的两侧点外，还包括一个delta参数，即公式(9-14)至(9-17)中用到的极小正实数 δ。
  - delta的默认值为10^{-4}，在Clipper的适配器类ClipperAdaptor中，将轮廓在转化中的默认精度设置为10^{-7}，显然，delta的值不能小于10^{-7}。
- **split**函数是SplitRegion类的核心，包括凹峰点寻找、凹峰点两侧点（即切分线段）寻找、切分矩形构造、裁剪区域切分等过程。
  - 最后一步中，将输入多边形列表以PT_SUBJECT类型添加至clipper，而将切分矩形列表以PT_CLIP类型添加至clipper，然后调用Execute函数并输入CT_DIFFERENCE进行布尔差运算，最终得到分割结果。
  - 需要说明的是，如果输入区域本来就是一个单连通区域，那split函数直接返回原输入区域多边形。
- 全局接口函数**splitRegion**，它的输入为待分区的平面多边形列表polygons和adjustPolyDirs，其中第二个参数的作用是保证输入的多边形方向符合规定，函数的输出为已分区的单连通多边形列表。

对填充区域进行分区的目的是
为了更方便地连接区域内的填充线！

4. 平行路径生成与连接

- **主要步骤**

在解决了**填充线的生成**和**填充区域的分区**问题之后，平行路径的生成与连接问题已经变得非常简单，只需在路径的生成流程上做一些控制即可，其主要步骤如下：

- **步骤1.** 输入表示待填充区域的平面多边形序列、填充间距及填充角度。
- **步骤2.** 根据填充角度对多边形序列进行顺时针旋转，得到新多边形序列。
- **步骤3.** 对新多边形序列进行分区，生成一系列单连通区域。
- **步骤4.** 根据填充间距在各个单连通区域内生成填充线。
- **步骤5.** 将各个单连通区域内的填充线段根据zig-zag的连接方式连成一体。
- **步骤6.** 根据填充角度对得到的每条连接路径做逆时针旋转，生成平行路径。
- **步骤7.** 输出平行路径序列。

4. 平行路径生成与连接

- **代码实现**——GenDpPath类：实现平行路径生成与连接

GenDpPath.py	方向平行填充路径生成与连接	9-12

```
1   class GenDpPath:                                        # 定义GenDpPath类
2       def __init__(self, polygons, interval, angle):      # 输入多边形列表、间距、角度
3           self.polygons, self.interval, self.angle = polygons, interval, angle
4           self.splitPolys = []                            # 保存分区得到的单连通多边形
5       def generate(self):                                 # 定义平行路径核心函数
6           rotPolys = GeomAlgo.rotatePolygons(self.polygons, -self.angle)          # 旋转输入多边形
7           self.splitPolys = splitRegion(rotPolys)         # 分区生成单连通区域
8           ys = self.genScanYs(rotPolys)                   # 生成扫描高度
9           paths = []                                      # 保存临时已连接路径
10          for poly in self.splitPolys:                    # 遍历每个单连通区域
11              segs = genHatches([poly], ys)               # 在单连通区域内生成填充线
12              if len(segs) > 0:
13                  path = self.linkLocalHatches(segs)      # 连接单连通区域内填充线段
14                  paths.append(path)
15          return GeomAlgo.rotatePolygons(paths, self.angle)   # 旋转生成的连接路径列表
16      def genScanYs(self, polygons):                      # 定义扫描高度生成函数
17          ys, yMin, yMax = [], float('inf'), float('-inf')
18          for poly in polygons:                           # 首先找出所有多边形的最低点
19              for pt in poly.points:                      # 和最高点
20                  yMin, yMax = min(yMin, pt.y), max(yMax, pt.y)
21          y = yMin + self.interval                        # 然后根据填充间距生成高度
22          while y < yMax:                                 # 列表
23              ys.append(y)
24              y += self.interval
25          return ys
26      def linkLocalHatches(self, segs):                   # 定义链接单连通区域内的
27          poly = Polyline()                               # 填充线函数
28          for i, seg in enumerate(segs):                  # 遍历列表，获取序号和元素
29              poly.addPoint(seg.A if (i%2==0) else seg.B) # 采用zig-zag的连接方式，
30              poly.addPoint(seg.B if (i%2==0) else seg.A) # 头和头连，尾和尾连
31              poly.endPoint().w = 1                       # 标记连接线起点w为1
32          return poly
33  def genDpPath(polygons, interval, angle):               # 定义GenDpPath类的全局接口
34      return GenDpPath(polygons, interval, angle). generate()   # 函数
```

4. 平行路径生成与连接

- **代码实现——GenDpPath类**
- GenDpPath类的输入包括：多边形列表polygons、填充间距interval以及填充角度angle。它的核心流程被封装在**generate**函数中。
- 首先对输入多边形根据填充角度进行了顺时针旋转。
  - 考虑到旋转多边形是一个通用功能，后面流程可能还会用到，在GeomAlgo模块中定义了一个**rotatePolygon**s函数：

GeomAlgo.py | **添加rotatePolygons函数，在XY平面内对多边形绕指定点进行旋转** | **9-13**

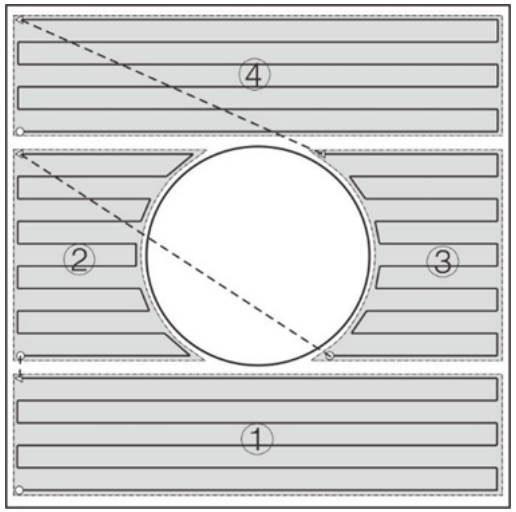
```
1  def rotatePolygons(polygons, angle, center = None):      # 定义旋转多边形函数
2      dx = 0 if center is None else center.x               # 旋转中心X坐标，默认为0
3      dy = 0 if center is None else center.y               # 旋转中心Y坐标，默认为0
4      mt = Matrix3D.createTranlsateMatrix(-dx, -dy, 0)    # 构造平移矩阵，matrix to
5      mr = Matrix3D.createRotateMatrix('Z', angle)        # 构造旋转矩阵，旋转轴为Z轴
6      mb = Matrix3D.createTranlsateMatrix(dx, dy, 0)      # 构造反向平移矩阵，matrix back
7      m = mt * mr * mb                                    # 变换矩阵乘积
8      newPolys = []
9      for poly in polygons:                               # 遍历，对每个多边形变换
10         newPolys.append(poly.multiplied(m))
11     return newPolys                                     # 返回变换后矩阵序列
```

如果旋转中心center为None，则默认旋转中心为坐标原点。

  - **rotatePolygons**的作用是计算获得XY平面上的多边形绕指定点旋转后的新多边形，其输入包括多边形序列polygons、旋转角度angle（以弧度为单位，正角度表示逆时针旋转）以及旋转中心点，输出为旋转后的新多边形序列。
- 对旋转后的rotPolys，调用splitRegion函数将其分割成一系列单连通多边形splitPolys。
  - 需要说明的是，这里将分区得到的单连通多边形splitPolys保存为了GenDpPath类的属性，目的是方便后续统计指定填充角度下的分区个数。
- 为了方便生成扫描高度列表ys，在类中又定义了一个**genScanYs**函数。
- 对每个单连通多边形和高度序列ys，将其输入**genHatches**函数以生成填充线段。

4. 平行路径生成与连接

- **代码实现——GenDpPath类**
- 将各个单连通区域内的填充线段分别输入**linkLocalHatches**函数，它的作用是采用zig-zag的连接方式将单连通区域内的填充线段连成一体，生成一条多段线路径。
  - 对每两条填充线段之间的连接线段，标识其起点的**w值为1**，表明这是一段空走路径，这个标记将在后续生成打印代码时用到。
- 最后，对各个单连通区域内得到的已连接平行路径，调用**rotatePolygons**函数将其逆时针旋转回原多边形内部。
- 在GenDpPath模块的结尾，定义了一个GenDpPath类的**全局接口函数genDpPath**，以方便后续使用该模块功能。
  - 它的输入参数和GenDpPath类的输入参数一样，包括多边形列表、填充间距及填充角度，输出为zig-zag平行路径列表。



单连通区域之间的全局路劲连接

- 如图，尽管已经对各个单连通区域内的填充线段进行了连接，但从全局的角度来讲，并未对各个单连通区域间的平行路径进行连接，而在实际应用中，它们是需要进一步被连成一体的。
- 从路径优化的角度来说，和上一章介绍的各条已连接轮廓路径的连接问题类似，单连通区域间平行路径的最短连接问题也是一个“**旅行商**”问题，而且得到的连接路径在实际打印（或加工）过程中只能通过“**跳刀**”的方式实现。

5. 最少跳刀填充方向优化

$$(\mathbf{v}_k \times \mathbf{v}_{k+1}).\mathrm{dz} < 0, \quad (9\text{-}9)$$

$$(\mathbf{v}_k \times \mathbf{v}_s).\mathrm{dz} \cdot (\mathbf{v}_{k+1} \times \mathbf{v}_s).\mathrm{dz} \le 0 \quad (9\text{-}12)$$

$$N_s = 2N_p + 1, \quad (9\text{-}13)$$

回顾：

在对多边形区域进行分区时，最终得到的单连通区域的个数和多边形上凹峰点的个数相关，它们之间的具体关系可由式(9-13)表示，即分区个数等于两倍的凹峰点个数加1。

- 根据式(9-9)可知，多边形中凹顶点的个数仅和多边形自身形状有关，而和填充方向无关。
- 根据式(9-12)可知，多边形中峰值顶点个数不仅和多边形自身形状有关，而且和填充方向有关。
- 考虑到多边形形状不可改变，因此，多边形中凹峰点个数仅和填充方向有关，即分区得到的单连通区域个数仅和填充方向有关。

对平行填充路径来说，显然，分区时得到的单连通区域的个数越少，后续的路径连接就越简单，需要的“跳刀”次数就越少。如果仅考虑单连通区域间平行路径的连接问题，那么跳刀次数（N_j）和单连通区域个数（N_s）的关系为：

$$N_j = N_s - 1, \quad (9\text{-}18)$$

小结：

① 对一个给定多边形区域，**跳刀次数也仅和填充方向有关**。

② 理论上，当填充角度从0~360度（考虑角度冗余情况，事实上是0~180度）变化时，必然存在一个最优的角度值，使得沿着这个角度对输入多边形区域进行分区时，生成的单连通区域个数最少，即跳刀次数最少。

任务：以减少跳刀次数为目标，找出最优填充角度。

5. 最少跳刀填充方向优化

- **目标函数建立**

目标函数就是所**关心的目标**（某一变量）和**相关因素**（某些变量）的函数关系。

- 如果目标只有一个，称这种优化问题为单目标优化；
- 如果目标有多个，称这种优化问题为多目标优化。

对于跳刀优化问题，它的目标是减少跳刀次数，影响因素是填充角度。显然，这是一个单目标、单因素的优化问题，其目标函数可以写成：

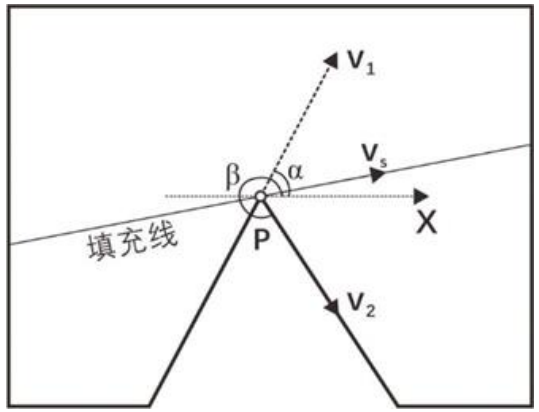
$$J = f(\theta), \qquad (9\text{-}19)$$

其中J为跳刀次数，θ为填充角度。

- 由于问题本身特性，无法给出函数f(θ)代数解析式。但是，对于某一固定多边形区域，在填充角θ已知的情况下，可以计算出分区个数、进而计算出跳刀次数的。即对目标函数式(9-19)，由θ到J的映射尽管很复杂，且无法表达成解析式，但映射过程却是非常明确的。
- 理论上，对一般单目标优化问题，求解其最优解的过程就是求解目标函数导数为0时的解。
- **对跳刀次数的优化问题，无法采用求导的方式求解。**

5. 最少跳刀填充方向优化

- **跳刀优化计算原理**
- **直观思路**：对目标函数进行正向计算，即直接在0~180度范围内遍历填充角度θ，计算每个θ对应的跳刀次数，然后从中找出跳刀次数最少时对应的填充角θ值。
  - 这种思路很简单，即对θ在0~180度范围内进行均匀细分取值，对每个细分角度θ_i，首先对输入多边形区域旋转顺时针θ_i角度，然后调用SplitRegion类中的**findTurnPoints**函数计算并统计细分角度θ_i下的凹峰点个数，最后再计算相应跳刀次数。 ——**为什么要旋转?**
- **存在问题**：实现以上思路不需要花费太多时间，但这种方法需要大量遍历，**计算效率不高**。
  - 以上思路的计算量主要取决于**细分精度**，比如当细分精度为0.01度时，将得到18000个细分角度，即要对原始多边形进行18000次旋转操作，且对每次旋转后的多边形，对其各个顶点，要进行18000次遍历。由此可非常直观地感受到其中计算量。
  - 在实际应用中，可能没必要将角度的细分精度设置得那么高，如细分精度为0.1度已足够高。



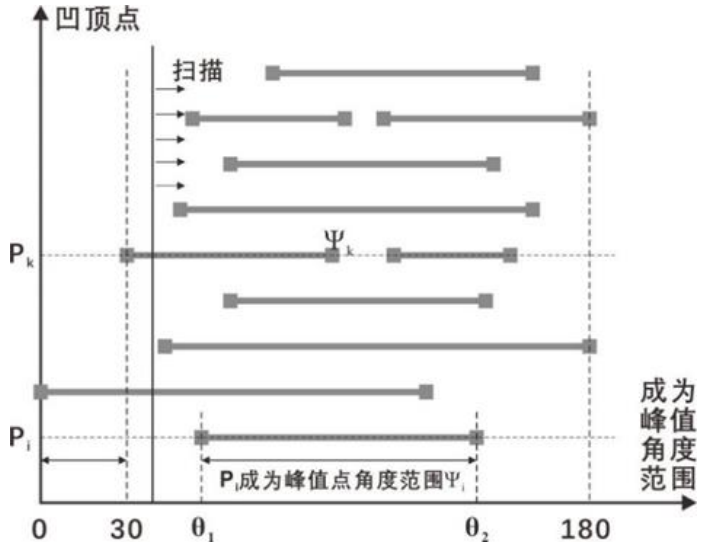
凹顶点成为峰值点的切线角度范围

对原始多边形的旋转操作可能并非必要!

- 如图，已知P为多边形上的一个凹峰点，$\mathbf{v_1}$、$\mathbf{v_2}$分别为与P相邻两边的方向向量，$\mathbf{v_s}$为扫描线（或填充线）方向向量。
- 根据峰值点定义，$\mathbf{v_1}$、$\mathbf{v_2}$应该分别在$\mathbf{v_s}$两侧。反过来，在向量$\mathbf{v_1}$、$\mathbf{v_2}$方向固定但$\mathbf{v_s}$方向可变的前提下，只要$\mathbf{v_s}$落在$\mathbf{v_1}$、$\mathbf{v_2}$之间，那么相对$\mathbf{v_s}$来说，P点就是一个峰值点。
- 据此，可反算出顶点**P成为峰值点时填充方向$\mathbf{v_s}$的角度范围**，称这个范围为P的**切线角度范围（或切线角度）**，记为Ψ。
- 对输入多边形上的任意凹顶点P_i，可找出它的切线角度范围Ψ_i。

5. 最少跳刀填充方向优化

- **跳刀优化计算原理**
- **问题转化：**如果把多边形上所有凹顶点的切线角度范围都绘制到同一个坐标系下，将得到一系列水平线段。
  - 每条线段表示一个凹顶点的切线角度范围，其中线段起点（左端点）表示角度范围上限，线段终点（右端点）表示角度范围下限。
  - 在图中，可构造一条竖直直线，使其在0~180度范围内扫描。在某个特定扫描角度下，如果直线和某一线段相交，则说明该线段对应的凹顶点在这个扫描角度下是一个峰值点。
  - 当直线移动时，统计每个位置上直线和线段的交点个数，理论上，可以**找出直线和线段交点数量最少时的扫描角度值（或范围）。**



根据多边形所有凹顶点切线角度确定最少跳刀填充角度

- 在这个扫描角度范围内（图中是0~30度），峰值点数量最少，凹峰点数量也最少，在对输入多边形区域进行平行填充时，如果填充角度也落在这个角度范围内，那可以得到最少的分区数量和最少的跳刀次数。
- 这个角度范围就是需要寻找的最少跳刀填充角度。

小结：

① 想要得到最少跳刀次数，对输入多边形的旋转操作确实是不必要的；

② 确定多边形上**每个凹顶点的切线角度**是寻找最少跳刀填充角度的基础。

5. 最少跳刀填充方向优化

- **顶点切线角度确定**

问题

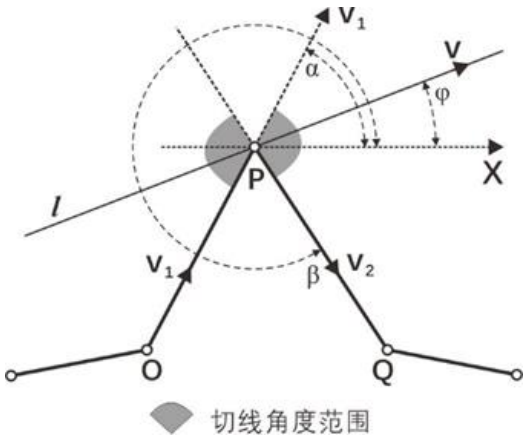
对连续可导轮廓上的某一点来说，它的切线方向固定，切线角度必然是一固定数值；但对离散轮廓上的某一顶点来说，它的切线角度是什么？

切线角度定义

如图，对XY平面任意离散轮廓上的一个顶点P，如果某一直线*l*经过顶点P但**并不“穿过”由3点组成的局部轮廓OPQ**，那么定义直线*l*为离散轮廓顶点P的切线，并且定义此时直线*l*和坐标系X轴正向的夹角为切线角度。

根据以上定义，可以推断：

- 首先，离散顶点的切线存在与否和顶点本身的**凹凸属性无关**，凹顶点可以有切线，凸顶点同样也可以有切线。
- 其次，当直线*l*和X轴夹角在一定范围内变动时，*l*都可能是该顶点的切线，因此，**离散顶点的切线角度表示的事实上是一个角度范围**，记这个范围为Ψ。



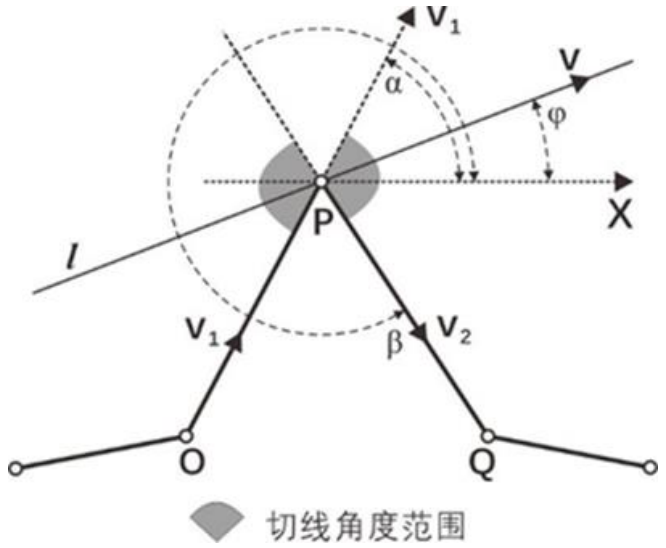
离散点切线及切线角度定义

- 如图，记顶点P两邻边方向向量分别为$\mathbf{v}_1$和$\mathbf{v}_2$，直线*l*的方向向量为$\mathbf{v}$，又记$\mathbf{v}_1$、$\mathbf{v}_2$、$\mathbf{v}$和X轴正向的夹角分别为α、β、φ。
- 考虑到$\mathbf{v}_1$、$\mathbf{v}_2$可能指向坐标系4个象限中的每个象限，因此夹角α、β的取值范围均为0~360度，即α、β∈[0, 360)。
- **任务**：在夹角对α、β已知的前提下，确定顶点P的切线角度范围Ψ。

5. 最少跳刀填充方向优化

- **顶点切线角度确定：Ψ的取值范围定性讨论**

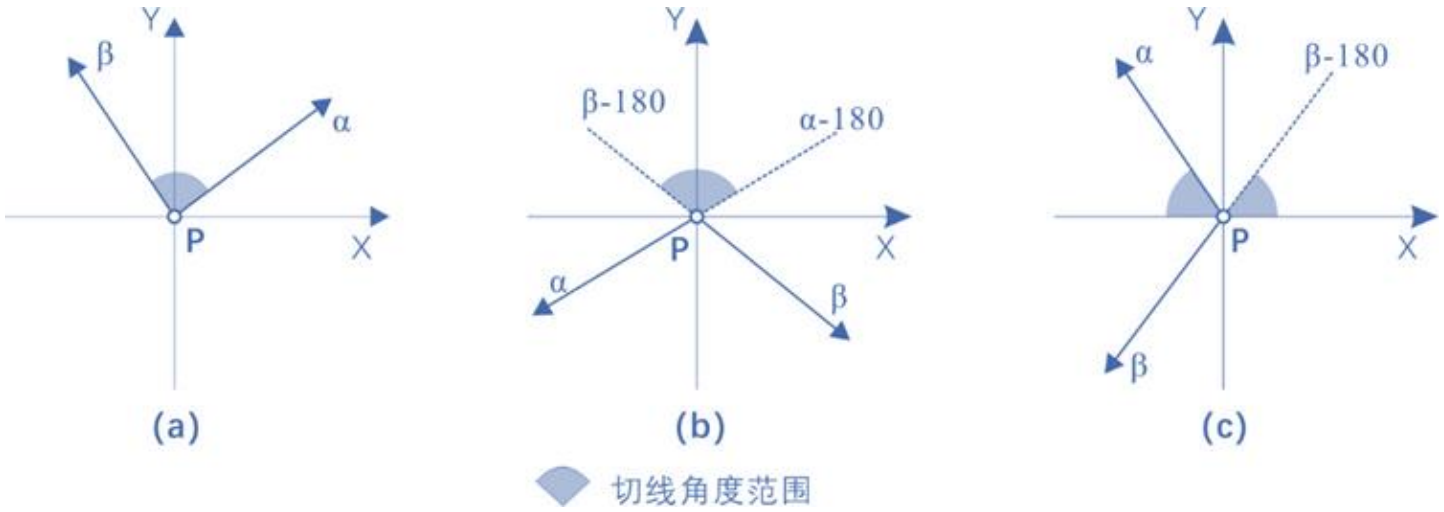
- 理论上，直线 *l* 的方向向量**v**可指向平面坐标系4个象限中的任一个象限，因此，φ的取值范围应该是0~360度。
- 但是，考虑到在过同一顶点的前提下，当直线 *l* 的倾斜角度分别为φ或(φ + 180)时，它们在平面上是完全重合的。
- 简单起见，这里不妨规定φ的定义域为0~180度，即φ ∈ [0, 180)。根据以上规定，相应的，可以得出，离散顶点切线角度范围Ψ ⊆ [0, 180]，即Ψ 的上下限也应落在0~180度之间。



离散点切线及切线角度定义

5. 最少跳刀填充方向优化

- **顶点切线角度确定：Ψ的取值范围定量讨论**
- 如图，夹角α、β分别为$\mathbf{v}_1$、$\mathbf{v}_2$和坐标系X轴正向的夹角，其取值均在0~360度内，在数值上，考虑到α可能大于β，也可能小于β，简单起见，不妨设α < β（如果实际上α > β，则对它们的数值进行交换）。据图不难得出Ψ取值的以下3种情况：
  - **情况1**、如图a，如果β ≤ 180，即α < β ≤ 180，那么Ψ为[α, β]；
  - **情况2**、如图b，如果α ≥ 180，即β > α ≥ 180，那么Ψ为[α-180, β-180]；
  - **情况3**、如图c，如果α ≤ 180且β ≥ 180，则令α' = α，β' = β – 180，如有必要，交换α'、β'值，使得α' < β'，那么Ψ为[0, α'] ∪ [β', 180)。
- 在情况1和2中，Ψ的取值均为单段连续区域，但在情况3中，Ψ的取值为两段连续区域。
- 无论哪种情况，对离散轮廓上的任意顶点P，只要其两邻边的方向已知，即夹角对α、β已知，都能确定其切线角度范围Ψ。



不同情况下离散点切线角度确定

5. 最少跳刀填充方向优化

- **最少跳刀优化实现：凹顶点切线角度计算实现**

- 寻找输入多边形上的所有**凹顶点**，并计算其相邻两边方向向量和坐标系X轴的**夹角对α、β**。
- 为此，在FindOptFeedDir模块中封装一个名为**_findAnglePairs**的全局函数中，作用是计算夹角对α、β。函数名中下划线前缀表示这不是一个对外的接口函数，代码如下：

FindOptFeedDir.py 最少跳刀优化模块，凹顶点夹角对α、β计算函数 9-14

```
1   def _findAnglePairs(polygons, adjustPolyDirs):      # 定义凹顶点夹角对计算函数
2       if adjustPolyDirs:                              # 如有必要，调整多边形方向
3           adjustPolygonDirs(polygons)                 # 使其符合“外逆内顺”的规定
4       anglePairs = []                                 # 用于保存夹角对列表
5       for poly in polygons:                           # 遍历多边形
6           for i in range(poly.count()-1):             # 遍历多边形顶点
7               pts = poly.points
8               v1 = pts[-2 if (i==0) else (i-1)].pointTo(pts[i])  # 当前顶点前边方向向量v1
9               v2 = pts[i].pointTo(pts[i + 1])         # 当前顶点后面方向向量v2
10              if v1.crossProduct(v2).dz < 0:          # 判断当前顶点是否是凹顶点
11                  a = radToDeg(v1.getAngle2D())       # 向量v1和X轴夹角a，0~360度
12                  b = radToDeg(v2.getAngle2D())       # 向量v2和X轴夹角b，0~360度
13                  a, b = min(a, b), max(a, b)         # 强制使：a < b
14                  anglePairs.append((a, b))           # 添加夹角对(a, b)到anglePairs
15      return anglePairs
```

非常重要！

转化为角度保存

- findAnglePair函数的输入为多边形列表polygons和布尔型变量adjustPolyDirs，如果adjustPolyDirs为True，则自动调整多边形的方向，使其满足**“外逆内顺”**的规定。这点非常重要，如果不满足，多边形上各顶点的凹凸特性将完全相反。
- 此外，在获得了当前顶点两邻边的方向向量之后，使用**getAngle2D**函数计算边向量和X轴的夹角，注意，getAngle2D函数返回值的范围为0~2π，我们需要使用Utility模块中的radToDeg函数将弧度转化为度数单位，使最终得到的角度对a、b取值落在0~360度范围内，方便理解。
- 最后，在_findAnglePair函数中，强制交换了得到了a、b值，使其满足a < b。

5. 最少跳刀填充方向优化

- **最少跳刀优化实现：扫描角度表**

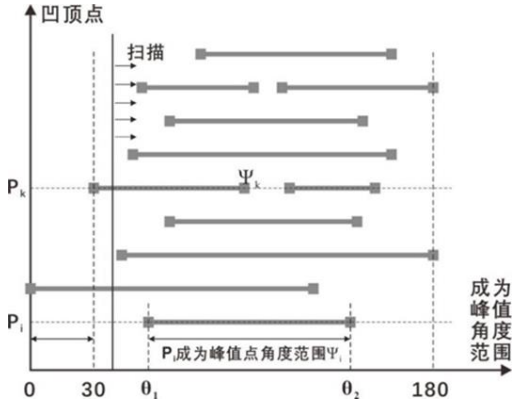
目标：寻找填充角度（或范围），使得多边形上的凹顶点尽量少地成为峰值点，即使得填充线尽量少地成为凹顶点的切线，即尽量减少图中竖直扫描线和表示凹顶点角度范围的线段的交点。

从算法实现的角度来说，很难构造这样的一条竖直扫描线，可以在0~180度范围内连续扫描，进而精确地找出最少跳刀角度范围。

——**这应该仍然是一个开放的问题。**

一种可行思路（对角度进行离散）：

- ✓ 对扫描范围进行离散，如将扫描角度间距设置为0.1度时，可得约1800个离散扫描角度，进而构造一个**扫描角度表**。
- ✓ 对角度表中的每个离散角度ω，关联一个变量t，用于标识在该角度下竖直扫描线和凹顶点切线角度（线段）的交点个数。
- ✓ 构建一个**双重循环**，首先遍历角度表，对每个扫描角度，构建对应扫描线，然后遍历**凹顶点切线角度**（线段）。如果某个切线角度和当前扫描线相交，则对当前扫描角度的变量t累加1。
- ✓ 在完成上述循环后，统计角度表中各个离散角度的变量t值，找出最少跳刀填充角度。



- 以上循环中，**首先遍历角度表，然后遍历凹顶点切线角度**，事实上，只要交换下顺序，就可大幅降低遍历次数。
- 对某个切线角度Ψ，设其上限和下限分别为φ_{min}和φ_{max}，即$\Psi = [\varphi_{min}, \varphi_{max}]$，这里暂时不去讨论Ψ被分为两段的情况。在图中，$\varphi_{min}$和$\varphi_{max}$就是切线角度（线段）的两个端点，根据这两个端点和扫描角度的离散规则，不难计算**出落在两端点间的所有离散扫描角度，此时只需对这些扫描角度对应的变量t累加1即可**。显然，以上方法同样适用于切线角度Ψ被分为两段的情况。
- 根据以上思路，只需对凹顶点切线角度使用一次遍历，就能最终找出最少跳刀填充角度。——**需要数据结支撑。**

5. 最少跳刀填充方向优化

- **最少跳刀优化实现：扫描角度表**

- 在Python中，可使用**字典类**来构建扫描角度表。在字典中，键为角度，值为交点个数，即上述变量t。在FindOptFeedDir模块中，将角度表的构造及初始化函数命名为**_initAngleTable**，它的具体实现代码如下：

FindOptFeedDir.py	最少跳刀优化模块，扫描角度表初始化函数	9-15

```
1  def _initAngleTable(digit):                  # 角度表构建函数，digit为小数位数
2      angleTable = {}                          # 角度字典
3      delta = math.pow(10, -digit)             # 角度增量
4      angle = 0.0
5      while angle < 180:                       # 在0~180度范围内，创建字典键
6          angleTable[angle] = 0                # 初始化每个角度下的交点数为0
7          angle = round(angle + delta, digit)  # 保证角度小数位数为指定值
8      return angleTable                        # 返回角度表
```

- 以上函数只有一个输入参数digit，表示角度表中各个角度保留的小数位数，即角度精度。
  - 当digit 值为1时，表示保留1位小数，此时角度表中角度分别为：0、0.1、0.2、…、179.9，共1800个角度值（注意，0和180度表示填充线重合，故忽略180度）；
  - 当digit值为0时，表示保留0位小数，此时角度表中角度分别为：0、1、2、…、179，共180个角度值。
- 初始情况下，在角度表中，每个角度对应的交点个数为0；后面就是需要从角度表中找出相交次数最少的角度值，并将它作为最少跳刀填充角度。
- _initAngleTable函数使用**round**函数确保角度表中的各角度值保留的小数位数是可控的。
  - 这一点非常重要，因为在后续代码中，正是根据可控的小数位数在角度表（字典）中查询相应角度的交点个数的。试想：如果小数位数不可控，尽管数值相似，但可能根本无法从角度表中找到目标键值。

5. 最少跳刀填充方向优化

- **最少跳刀优化实现：findOptFeedDir**

- 结合以上讨论的Ψ取值的3种情况给出最少跳刀填充角度的计算函数**findOptFeedDir**，其具体实现代码如下：

FindOptFeedDir.py | **最少跳刀优化模块，寻找最少跳刀填充角度函数** | **9-16**

```
1  def findOptFeedDir(polygons, digit = 0, adjustPolyDirs = False):      # 定义函数
2      anglePairs = _findAnglePairs(polygons, adjustPolyDirs)    # 获取凹顶点角度对列表
3      angleTable = _initAngleTable(digit)                       # 获取扫描角度表（字典）
4      delta = math.pow(10, -digit)                              # 扫描角度增量
5      for a, b in anglePairs:                                   # 遍历角度对，获取切线角度
6          if b <= 180:                                          # 情况1：[a, b]
7              key = round(a + delta, digit)                     # 从切线角度上限开始
8              while key <= round(b - delta, digit):             # 遍历直至切线角度下限
9                  angleTable[key] += 1                          # 当前角度上的交点个数累加1
10                 key = round(key + delta, digit)               # 角度自增delta
11         elif a >= 180:                                        # 情况2：[a-180, b-180]
12             key = round(a - 180 + delta, digit)               # 同情况1
13             while key <= round(b - 180 - delta, digit):
14                 angleTable[key] += 1
15                 key = round(key + delta, digit)
16         elif a <= 180 and b >= 180:                           # 情况3：[0, a']∪[b', 180)
17             b = b – 180                                       # 先对a、b值进行调整
18             a, b = min(a, b), max(a, b)
19             key = round(delta, digit)                         # 前半段 [0, a']
20             while key < a:
21                 angleTable[key] += 1
22                 key = round(key + delta, digit)
23             key = round(b + delta, digit)                     # 后半段 [b', 180)
24             while key < 180:
25                 angleTable[key] += 1
26                 key = round(key + delta, digit)
27     return angleTable                                         # 返回角度表
```

5. 最少跳刀填充方向优化

- **最少跳刀优化实现：findOptFeedDir**

- **findOptFeedDir**函数的输入参数包括polygons、digit以及adjustPolyDirs，默认情况下，设定digit为0，即角度精度为1度，adjustPolyDirs为False，即无需调整输入多边形方向。
- findOptFeedDir函数中，首先做了一些数据准备工作，包括调用_findAnglePairs函数生成多边形上所有凹顶点角度对列表anglePairs，以及调用_initAngleTable函数获取初始化扫描角度表angleTable。随后是函数主体，根据角度对中两角度的取值情况分3种情况讨论。
- 以第1种情况为例说明，第2及第3种情况类似：
  - 在第1种情况下，$b \le 180$，即$a < b \le 180$，此时切线角度的取值范围为[a, b]，需要做的就是对[a, b]范围内的各个扫描角度对应的交点个数加1。
  - 在代码中，对切线角度范围[a, b]进行了压缩，使其变成[a+delta, b-delta]，其中delta为角度表中角度增量（或称为角度精度、角度分辨率），这其实是一种偏保守或偏安全的做法，确保在此区间的角度值对应的直线都是相应顶点的切线。
  - 代码中使用了多次round函数进行四舍五入，确保计算得到的小数位数可控且为指定值，这样做的目的是保证根据角度值查询角度表（字典）angleTable时不会出现找不到键值的情况。

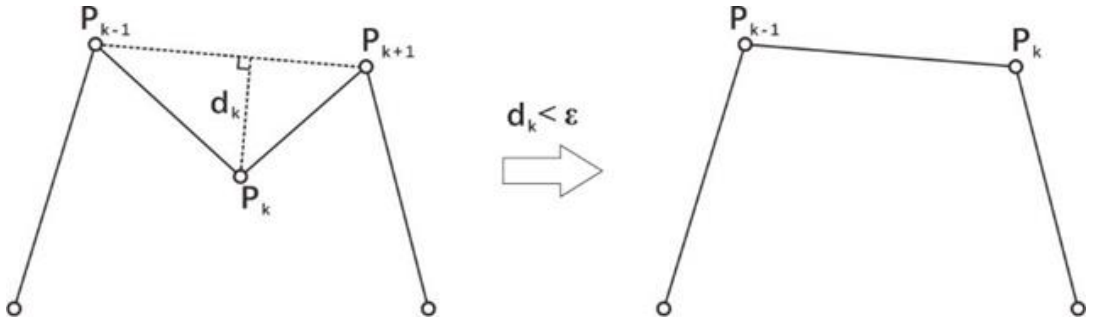
情况1、如图a，如果$\beta \le 180$，即$\alpha < \beta \le 180$，那么Ψ为$[\alpha, \beta]$；

情况2、如图b，如果$\alpha \ge 180$，即$\beta > \alpha \ge 180$，那么Ψ为$[\alpha-180, \beta-180]$；

情况3、如图c，如果$\alpha \le 180$且$\beta \ge 180$，则令$\alpha' = \alpha$，$\beta' = \beta - 180$，如有必要，交换α'、β'值，使得$\alpha' < \beta'$，那么Ψ为$[0, \alpha'] \cup [\beta', 180)$。

5. 最少跳刀填充方向优化

- **最少跳刀优化实现：findOptFeedDir**
- 在完成3种情况讨论后，角度表angleTable中每个角度对应的交点个数都应被赋值，函数返回。理论上，只要遍历返回的角度表，就能从中找出交点个数最少对应的角度值。只要将这些角度中的任意一个角度输入**genDpPath**函数，就能生成最少跳刀填充路径。
- 需要指出的是：通过事先移除输入多边形上冗余精度点的方式可进一步对findOptFeedDir模块进行优化。
  - 如图，对多边形上的3个连续顶点P_{k-1}、P_k和P_{k+1}，如果P_k到线段$P_{k-1}P_{k+1}$的距离d_k小于一个极小的正实数ε，那么定义P_k是一个冗余精度点。
  - 根据定义，多边形上的平角点显然是一个冗余精度点。此外，只要d_k足够小，删除P_k并不会对原多边形轮廓造成什么影响，反而可以减少轮廓上的细微扰动，使多边形变得更加简洁。
  - 对findOptFeedDir模块来说，删除冗余精度的凹顶点是非常有意义的，可以减少代码9-14中需要计算的角度对，提升findOptFeedDir函数的鲁棒性。



多边形上冗余精度点删除

6. 平行路径生成与优化测试

- 通过前面介绍，得到了4个和平行填充路径生成相关的模块，包括：
  - **GenHatch** **扫描填充线生成**
  - **SplitRegion** **填充区域分区**
  - **GenDpPath** **平行路径生成**
  - **FindOptFeedDir** **最少跳刀填充方向优化**
- 考虑GenHatch、SplitRegion两个模块是GenDpPath的支撑模块，而FindOptFeedDir又是一个相对独立的模块，因此这里主要对**GenDpPath**和**FindOptFeedDir**两个模块展开测试。

6. 平行路径生成与优化测试



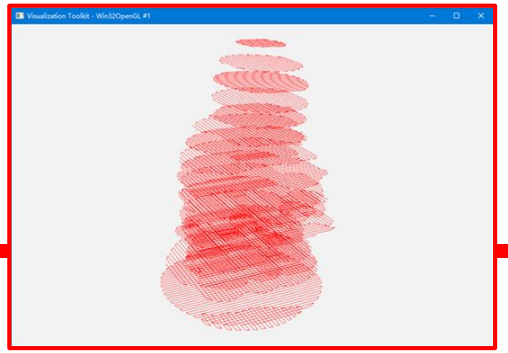
- **路径生成测试：测试代码**

- 首先对GenDpPath模块进行测试。考虑到之前已经得到了针对monk模型的不同切片厚度（分别为0.2mm、0.5mm、1.0mm、2.0mm）的4个SLC文件，这里直接采用这4个切片文件进行测试。

Test_genDpPath.py	平行路径生成测试：读取SLC文件，生成每平行填充路径	9-17

```
1  from GenDpPath import *                                  # 导入GenDpPath模块
2  interval = 3                                             # 设置填充路径间距
3  angle = 0                                                # 设置填充角度，单位：度
4  pathses = []                                             # 存放各切片层平行路径
5  layers = readSlcFile("D:\\monk at 1.0mm.slc")            # 读取SLC文件
6  start = time.clock()                                     # 计时开始
7  for i in range(len(layers)):                             # 遍历各切片层轮廓
8      print('dp, layer: %d / %d' % (i+1, len(layers)))     # 打印当前切片层信息
9      theta = degToRad(angle) if (i%2==1) else degToRad(angle+90)          # 正交填充角度
10     paths = genDpPath(layers[i].contours, interval, theta)       # 生成平行路径
11     pathses.append(paths)
12 end = time.clock()                                       # 计时结束
13 print("GenDpPath time: %f CPU seconds" % (end - start))     # 打印平行路径生成所需时间
14 va = VtkAdaptor()                                        # 调用VTK绘图
15 for layer in layers:                                     # 绘制切片轮廓，黑色
16     for contour in layer.contours:
17         va.drawPolyline(contour).GetProperty().SetColor(0, 0, 0)
18 for paths in pathses:                                    # 绘制平行填充路径，红色
19     for path in paths:
20         va.drawPolyline(path).GetProperty().SetColor(1, 0, 0)
21 va.display()
```

6. 平行路径生成与优化测试



• 路径生成测试

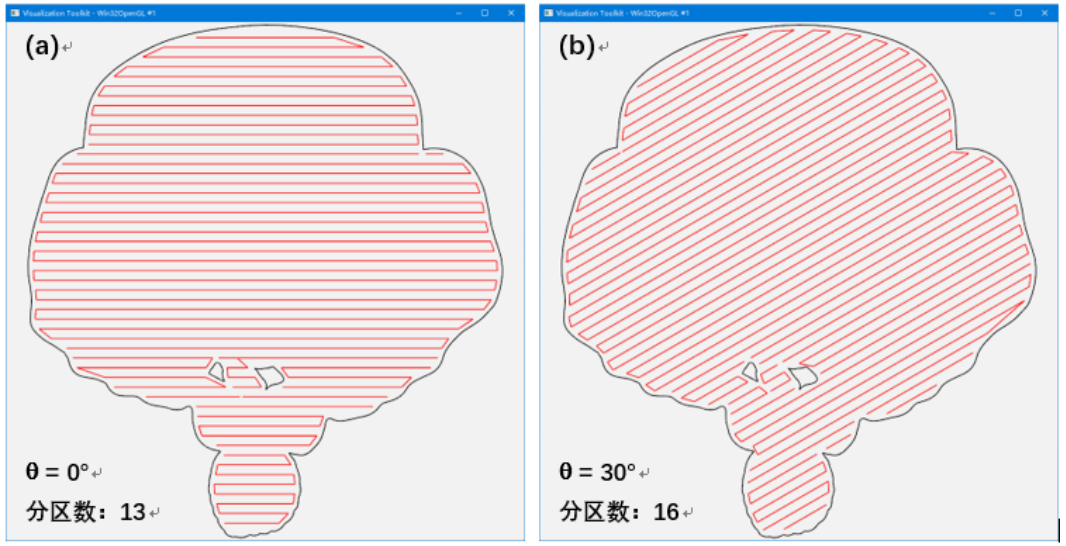
- 以上对平行路径生成的测试代码和上一章对轮廓路径生成的测试代码十分相似，只做了三处主要修改：
  - 一是设置了变量interval，这个变量的作用是设置填充线间距，这里我们设置间距为3mm；
  - 二是将genCpPath函数替换成了genDpPath；
  - 三是根据平行路径的特点在相邻上下两层之间设置了正交化填充角度，即如果上一层填充角度为0度，那么下一层填充角度就是90度，以形成上下层之间的编织效果。当然，上下层之间的填充角度差值是可以任意设置的。
- 如表，在切片厚度为0.2mm时，针对一个拥有超33万个面片的模型来说，生成平行路径的时间仅需13.5秒，这个时间远远少于针对相同SLC文件生成轮廓路径所需的时间（132.9秒），当然两者本没有什么可比性，但可从另一个侧面反映本章设计并实现的平行填充路径生成算法是具有实际应用价值的。
- 此外，如表所示，平行路径生成时间大致和切片厚度的倒数成正比关系。

模型	面片数	模型高度 (mm)	切片厚度 (mm)	层数	平行路径生成时间 (秒)
monk	336,974	175.5	2.0	87	1.3
			1.0	175	2.7
			0.5	351	5.5
			0.2	877	13.5

monk模型在不同切片厚度下的平行路径生成时间统计，填充间距3mm，测试主机CPU为intel Core i7-8700 @ 3.20GH

6. 平行路径生成与优化测试

- **路径生成测试**
- 下图清晰地显示了在某一指定切片上（切片厚度2mm，第43层）根据切片轮廓生成的平行填充路径。
  - 在左图中，设置填充角度为0度，最终生成了13条独立路径；
  - 在右图中，设置填充角度为30度，最终生成了16条独立路径。
- 可见，至少在这两个填充角度下，填充角为0度时得到的平行路径要优于填充角为30度时得到的平行路径。但在0~180度范围内，对图中轮廓，到底哪个（些）角度是最优填充角度，这还需要借助**FindOptFeedDir**模块找出。



monk模型某一层切片上的平行填充路径

6. 平行路径生成与优化测试

- **跳刀优化测试**

- FindOptFeedDir模块的核心为其同名函数**findOptFeedDir**，
  - 该函数包含3个输入参数，分别为：当前层轮廓列表polygons、小数位数digit（默认为0）以及表示是否需要调整输入轮廓方向的布尔型变量adjustPolyDirs（默认为False，即无需调整轮廓方向）。
  - 该函数输出为一个凹峰点关于填充角度的字典（角度表），其中字典的键表示0~179度的角度值，相邻角度间距为1度，字典的值表示填充线在相应角度下的凹峰点个数。
- 根据公式(9-13)，在已知轮廓凹峰点个数（Np）的情况下，可计算得到相应分区个数（2Np+1）。
- 在测试文件中创建创建两个函数：
  - 一个是**getAngleTable**函数，它的输入为指定切片层轮廓，功能是调用上述**findOptFeedDir**函数（使用默认参数）并打印计算得到的相应角度表，同时统计函数运行时间。
  - 作为对比，另一个为**getRegionTable**函数，这个函数的功能是使用**暴力方法计算分区表**，即首先对填充角在0~180度范围内细分取值（这里设置角度间距为1度），然后根据每个填充角对输入轮廓进行旋转，最后逐次调用SplitRegion模块中的splitRegion函数对输入轮廓进行实际分区并统计最终得到的分区数，存于分区表中。根据getRegionTable函数的实现原理，可以推断，它的运行效率必然不及getAngleTable。
- 在测试文件中创建的第三个函数为**getPathCount**，作用是对输入切片轮廓调用genDpPath函数，生成轮廓区域内的实际平行填充路径，最终统计并打印实际路径数量。理论上，对给定切片层轮廓，由genPathCount生成的实际路径数量应等于getRegionTable生成的分区数。

6. 平行路径生成与优化测试

- **跳刀优化测试：测试代码**

Test_findOptFeedDir.py	跳刀优化测试：输入切片轮廓，计算、打印角度表	9-18

```
1   def getAngleTable(polygons):
2       start = time.clock()
3       angleTable = findOptFeedDir(polygons, 0)
4       for key in angleTable.keys():
5           print(key, angleTable[key])
6       end = time.clock()
7       print("getAngleTable time: %f CPU seconds\n" % (end - start))
8   def getRegionTable(polygons):
9       start = time.clock()
10      regionTable = {}
11      for angle in range(0, 180, 1):
12          rotPolys = rotatePolygons(polygons, -degToRad(angle))  # 对轮廓旋转，反向
13          splitPolys = splitRegion(rotPolys)
14          regionTable[angle] = len(splitPolys)
15      for key in regionTable.keys():
16          print(key, regionTable[key])
17      end = time.clock()
18      print("getRegionTable time: %f CPU seconds\n" % (end - start))
19  def getPathCount(polygons, interval):
20      for angle in range(0, 180, 1):
21          paths = genDpPath(polygons, interval, degToRad(angle))
22          print(len(paths))
23  if __name__ == '__main__':
24      layers = readSlcFile("D:\\monk at 2.0mm.slc")
25      layer = layers[43]
26      getAngleTable(layer.contours)
27      getRegionTable(layer.contours)
28      getPathCount(layer.contours, 0.1)
```

6. 平行路径生成与优化测试

- **跳刀优化测试：结果统计**

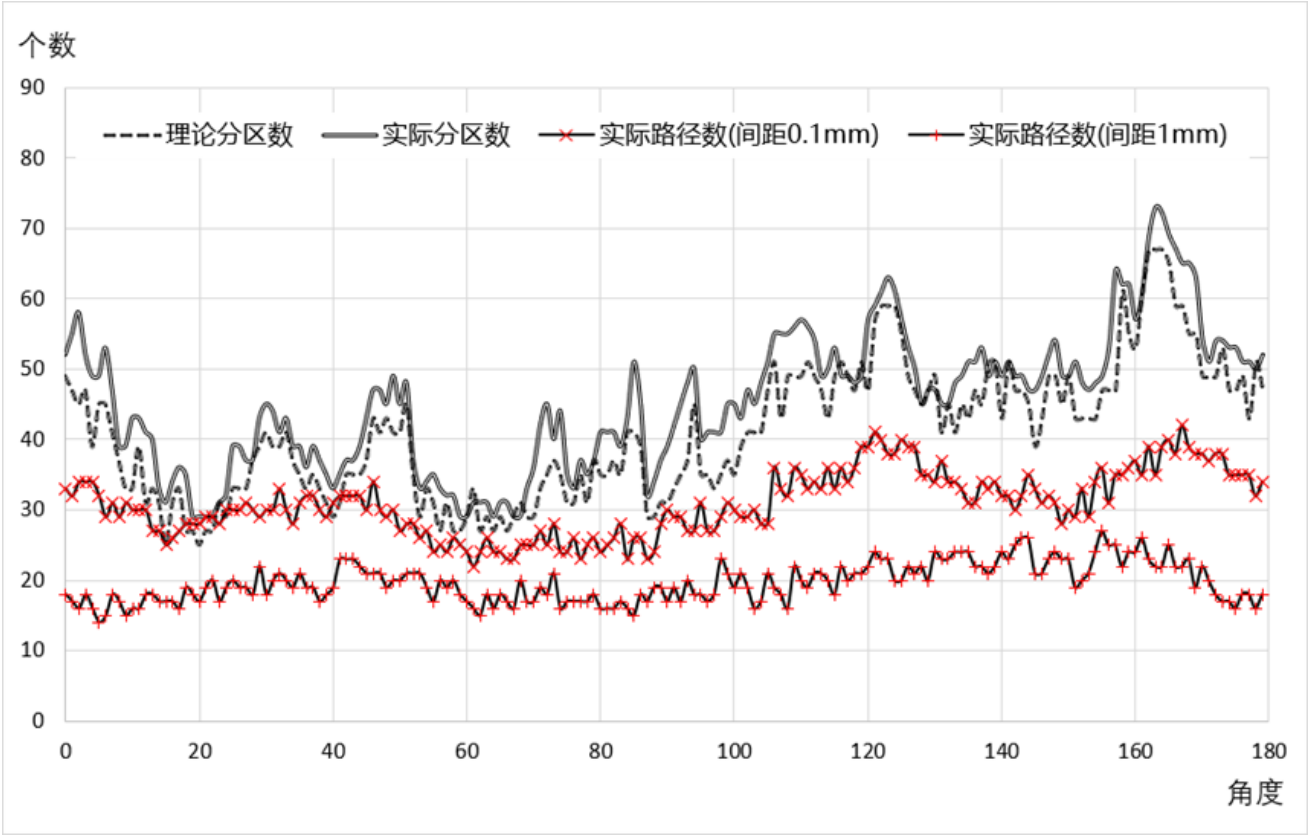
- 使用切片厚度为2mm的monk SLC文件，选取轮廓来自第43层切片。
- 对该层轮廓，依次调用**getAngleTable**、**getRegionTable**、**getPathCount**三个函数，下表各行分别给出了各个角度下的凹峰点个数、理论分区数、实际分区数以及实际路径数（路径间距为0.1mm和1.0mm），其中各角度下的理论分区个数通过凹峰点个数间接计算得到。

角度（度）	0	1	2	3	4	5	6	7	8	9	10	…	177	178	179
凹峰点个数 (N_P)	24	23	22	23	19	22	22	20	18	16	16	…	21	25	23
理论分区数 ($2N_p+1$)	49	47	45	47	39	45	45	41	37	33	33	…	43	51	47
实际分区数	52	55	58	52	49	49	53	47	39	39	43	…	51	50	52
实际路径数 (间距0.1mm)	33	32	34	34	34	32	29	31	29	31	30	…	35	32	34
实际路径数 (间距1.0mm)	18	17	16	18	16	14	15	18	17	15	16	…	18	16	18

monk模型某层切片轮廓在不同填充角度下的分区个数统计

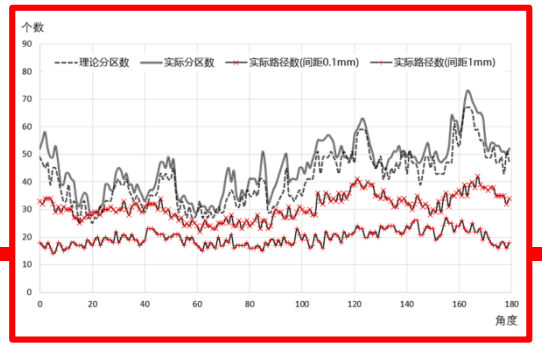
6. 平行路径生成与优化测试

- **跳刀优化测试：结果统计**
- 以上表中各行数据的可视化结果如图所示。在图中，从上往下各条曲线分别表示实际分区数、理论分区数、填充间距为0.1mm时的实际路径数以及填充间距为1mm时的实际路径数。



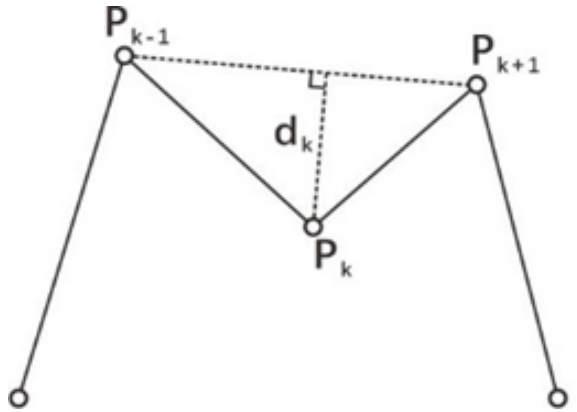
monk模型某层切片轮廓在不同填充角度下的分区数量及变化趋势

6. 平行路径生成与优化测试



• 跳刀优化测试：分析

- 根据上图，由getAngleTable函数间接计算得到的理论分区数和由getRegionTable直接得到的实际分区数在数值和曲线走势两个方面都基本吻合，最大相对偏差约为**20.1%**，平均相对偏差约为9.7%。当填充角度为15度时，理论分区曲线达到最低点，即分区数达到最小值，最小值为25，此时计算得到的实际分区数为31。
- 从图中还可以看到，实际分区曲线总是在理论分区曲线上方，即在同一填充角度下，实际分区数量总是大于理论分区数量，这是因为在findOptFeedDir函数中计算凹峰点数量时（参见代码9-16和图9.25），对切线角度范围[a，b]进行了压缩，使其变成[a+delta，b-delta]，其中delta为一正实数，这其实是一种**偏保守**的做法，导致部分处在临界角度边缘的凹峰点未被统计在列。
- 在图中，除了上述两条理论和实际分区数曲线之外，还绘制了两条实际路径数曲线，对应的填充间距分别为0.1mm（上）和1.0mm（下）。
  - 当填充间距为0.1mm时，曲线走势和上述分区数曲线大致相关，但实际路径数值明显小于分区数；
  - 当填充间距为1.0mm时，曲线无论在数值和走势上都已和分区数曲线基本无关，且当角度变化时，曲线在数值上趋近某一固定值（图中为20左右）。
  - 出现这种现象的原因是输入轮廓上的顶点在平面上排布较为致密，且在某些局部位置上出现了细小的锯齿形扰动（冗余精度点），如下图所示。对这种轮廓，在使用getAngleTable或getRegionTable计算分区数时，会将局部“锯齿”附近的区域分割并统计在内，导致分区数量上升。对于这些细小的分区，在生成平行填充路径时，如果填充间距大于区域本分的尺寸，则不会在分区内生成路径，这就是为什么对getPathCount函数来说，路径间距越大，输出的实际路径数越少的原因。



轮廓上的锯齿形扰动

本章小结

- 三维打印的常用填充路径包括轮廓路径和平行路径，这一章主要介绍的是**平行路径**的生成原理和方法。
  - 平行路径本质上是一系列平面平行填充线段，本章使用了两种方法生成这些平行填充线段，一是扫描线法，二是基于Clipper库的裁剪方法。考虑到裁剪法得到的平行填充线可能存在顺序混乱的问题，本章采用的是扫描线法生成平行填充线。
  - 对得到的填充线进行连接并生成路径时，需要对填充区域进行分区，针对该问题，提出了一种基于轮廓凹峰点的区域分区方法，在每个分区内，所有平行填充线都可以被首尾相连成zig-zag路径。
  - 最后，从减少平行路径上跳刀次数的角度，对平行路径进行了优化。
- 结合本章介绍的平行路径和上一章介绍的轮廓路径，可在每层切片轮廓内部生成完整的填充路径。





思考题

1. 设计一种算法，快速内报告平面线段（直线段、圆弧、椭圆弧、样条线段等）集合中的所有交点。
2. 在使用扫描线算法求解平行填充线的过程中，如何解决扫描线恰好穿过多边形轮廓顶点时的数值稳定性？
3. 在代码9-2中，分析语句if segs[i].yMin < y and segs[i].yMax >= y中等号是否取到对求交结果的影响。
4. 为什么选择平面坐标系原点作为多边形及填充线的旋转中心？是否可以选择其他点？
5. 在基于裁剪法的填充线生成过程中，为什么需要生成恰能覆盖输入轮廓区域的平行线段集合？
6. 尝试从源码层面探索由Clipper库裁剪得到的填充线在平面上的分布规律。
7. 在方向平行填充路径生成过程中，为什么需要对输入的填充区域进行分区？
8. 什么是凹峰点？如何求取平面多边形上的凹峰点？
9. 设计一种算法，考虑同一分区内的平行路径的两种连接方式，尝试以总路径最短为目标，连接不同分区中的平行填充路径。
10. 思考一种新的最少跳刀优化填充方向计算方法。
11. 除方向平行和轮廓平行两种填充方式外，设计一种新的打印区域填充方式。